



RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEUR ET DE LA RECHERCHE
SCIENTIFIQUE

UNIVERSITÉ DE TUNIS EL MANAR
École Nationale d'Ingénieurs de Tunis



RAPPORT DE FIN D'ANNÉE

Présenté en vue de l'obtention du
Diplôme National de Master
Spécialité : **Actuariat, Sciences des Données et
Contrôle Stochastique**

Élaborés par :

Zouaoui Ahmed Anas
Abbouz Hamza

Encadrée par :

Mme. Rabhi Anissa

Sujet du projet :

**Analyse comparative des modèles de séries
temporelles dans la gestion de la liquidité
bancaire**

Année Universitaire : **2024–2025**

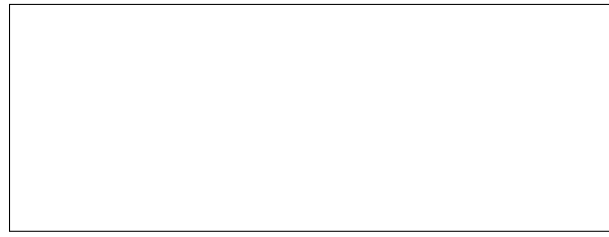
Remerciements

Nous tenons à exprimer nos sincères remerciements à Madame **Rabhi Anissa**, notre encadrante, pour sa bienveillance, sa rigueur scientifique et la qualité de son accompagnement tout au long de ce projet. Sa disponibilité, ses conseils pertinents et son encadrement constant ont grandement contribué à la réalisation de ce travail.

Nous exprimons également notre profonde gratitude à l'ensemble des enseignants du Master **Actuariat, Sciences des Données et Contrôle Stochastique** de l'École Nationale d'Ingénieurs de Tunis, pour la richesse de leurs enseignements et pour l'environnement intellectuel stimulant qu'ils nous ont offert tout au long de notre formation.

Ce projet de fin d'année, intitulé **Analyse comparative des modèles de séries temporelles dans la gestion de la liquidité bancaire**, nous a permis de mettre en œuvre les compétences acquises au fil de notre cursus, et d'approfondir nos connaissances dans un domaine au croisement de l'analyse quantitative et de la finance bancaire.

Enfin, nous remercions chaleureusement nos familles et nos proches pour leur soutien indéfectible et leurs encouragements constants.



Signature de l'encadrante : **Anissa Rabhi**

Resumé

Ce projet s'inscrit dans le cadre de l'obtention du diplôme de Master en Actuariat, Sciences des Données et Contrôle Stochastique Il vise à développer des modèles prédictifs de séries temporelles afin d'anticiper les besoins en liquidité dans le secteur bancaire. L'objectif principal est d'optimiser la stratégie de trésorerie en prévoyant les flux de retraits clients, notamment durant des périodes sensibles telles que le Ramadan ou l'Aïd.

Ce projet apporte une contribution concrète à la gestion proactive de la liquidité bancaire, en aidant les agences bancaires à ajuster leurs emprunts auprès de la banque centrale et à réduire les coûts liés aux excès ou manques de trésorerie.

L'étude se concentre sur une analyse comparative de modèles statistiques tels que ARIMA, SARIMA, ARCH, GARCH et des méthodes hybrides (comme l'association ARIMA-Random Forest). Cela inclut une étape d'analyse des données de transaction bancaire, un prétraitement (nettoyage, transformation de dates), une visualisation et une décomposition des séries temporelles pour déterminer tendance, saisonnalité et résidus.

L'évaluation des performances des modèles se fait sur la base de divers indicateurs (MSE, RMSE, R^2 ...), et leur aptitude à saisir les dynamiques temporelles est testée sur de véritables ensembles de données. La méthode utilisée comprend aussi des analyses statistiques (ADF, Ljung-Box, Jarque-Bera...) pour confirmer les suppositions de stationnarité et d'hétéroscédasticité.

Table des Matières

Introduction générale	9
1 Présentation du Contexte général du projet	10
1.1 Introduction	10
1.2 Le cadre du projet	10
1.3 Problématique	10
2 Théorie de la modélisation des Séries Temporelles	12
2.1 Introduction :	12
2.2 Définition d'une serie chronologique	12
2.3 Composantes d'une série chronologique	12
2.3.1 Tendance	12
2.3.2 Saisonnalité	13
2.3.3 Cycle	13
2.3.4 Résidu	13
2.4 Étude de la Stationnarité des Séries Chronologiques : Tests Unitaires et Application du Test de Dickey-Fuller	14
2.4.1 Stationnarité d'une série chronologique	14
2.4.2 Test unitaire	14
2.4.3 Test de dickey-Fuller	15
2.5 Bruit Blanc	19
2.5.1 Definition	19
2.5.2 Test de ljung-box	20
2.5.3 Test de jarque-bera	20
2.5.4 Test homoscedasticité et hétéroscedasticité	22
2.6 Décompositions d'une serie chronologique	22
2.6.1 Décomposition additive	22
2.6.2 Décomposition multiplicative	23
2.7 Les modèles des Séries Temporelles	23
2.7.1 ARMA	23
2.7.2 ARIMA	24
2.7.3 SARiMA	26
2.7.4 ARCH	27
2.7.5 GARCH	27
2.8 Random Forest	28
2.9 Métriques d'évaluation	28
2.9.1 Mean Squared Error (MSE)	28
2.9.2 Mean Absolute Error (MAE)	29
2.9.3 Root Mean Squared Error (RMSE)	29

2.9.4	Coefficient de détermination (R-squared)	29
3	Analyse et modélisation des Séries Temporelles	30
3.1	Introduction	30
3.2	Présentation des données	30
3.3	Environnement de travail	31
3.3.1	Python	31
3.3.2	Anaconda	31
3.4	Prétraitement des données	32
3.4.1	Nettoyage des données	32
3.5	Conversion et réindexation de la colonne de dates	32
3.6	Visualisation	33
3.6.1	compareaison de l'évolution des transactions d'une année à l'autre .	34
3.6.2	Analyse mensuelle de la distribution des montants	35
3.7	Décomposition de la série temporelle	35
3.7.1	Test ADF	38
3.7.2	Test stationnarité	40
3.8	Identification et Ajustement des Modèles ARIMA	40
3.8.1	Estimation des Paramètres ARIMA	40
3.8.2	Validation du modèle ARIMA	42
3.9	Prévision	45
	Conclusion et perspectives	53

Table des figures

2.1	la destribution asymptotique[6]	17
2.2	valeur tabulée de Dickey-Fuller[6]	19
2.3	Exemple d'une décomposition additive cette figure est importé de [10]	23
2.4	Exemple d'une décomposition multiplicative [10]	23
3.1	5 premières et 5dernières lignes de la base des données	30
3.2	description de la base de donnée	31
3.3	Logo Python	31
3.4	Logo Anaconda	31
3.5	Logo Jupyter NoteBook	32
3.6	resultat Code Python pour le nettoyage du data	32
3.7	resultat Code Python pour la conversion de la date	33
3.8	Visualisation de la serie chronologique	33
3.9	filtre une période spécifique 2020-2021	34
3.10	filtre une période spécifique 2021-2022	34
3.11	filtre une période spécifique 2022-2023	34
3.12	Distribution des montants par mois	35
3.13	Décomposition de la série temporelle selon le modèle additif	36
3.14	Décomposition de la série temporelle selon le modèle multiplicatif	37
3.15	Décomposition de la série temporelle en appliquant le logarithme sur le modèle multiplicatif	38
3.16	Resultat du test ADF	39
3.17	série après différenciation	39
3.18	resultat du test de stationnarité	40
3.19	ACF et PACF de la série différenciée	41
3.20	sélection de meilleur modèle pour déterminer la valeur de p	42
3.21	Validation du modele ARIMA	43
3.22	resultat du test ljung-Box	44
3.23	resultat du test Jaque-Bera	44
3.24	resultat du test hétéroscédasticité	45
3.25	Ajustement du modèle ARIMA sur l'ensemble d'entraînement	46
3.26	ARIMA Forecast	47
3.27	Prévision ARIMA(11,1,1)	48
3.28	Prévision Random Forest	49
3.29	Reconstruction de la série d'origine à partir des prédictions	50
3.30	Modèle combiné entre ARIMA(11,1,1) et Random Forest	52

Liste des codes

3.1	Code Python pour le nettoyage du data	30
3.2	Code Python pour le nettoyage du data	32
3.3	Code Python pour la conversion de la date	32
3.4	Code Python pour la période spécifique 2020-2021	34
3.5	Code Python pour la période spécifique 2021-2022	34
3.6	Code Python pour la période spécifique 2022-2023	34
3.7	Code Python pour la distribution des montants par mois	35
3.8	Code Python pour le modèle additif	36
3.9	Code Python pour le modèle multiplicatif	36
3.10	Code Python à l'application de logarithme	37
3.11	Code Python pour le test ADF	38
3.12	Code Python pour la différenciation	39
3.13	Code Python pour la différenciation	40
3.14	Code Python pour la différenciation	40
3.15	Code Python pour déterminer la valeur de p	41
3.16	Code Python pour validation modèle ARIMA	42
3.17	Code Python pour test de ljung-Box	43
3.18	Code Python pour test de Jaque-Bera	44
3.19	Code Python pour test d'Hétéroscédasticité	45
3.20	Séparation des données en ensembles d'entraînement et de test pour la prévision	46
3.21	code python pour comparer ARIMA Forecast et le réel	46
3.22	Restauration de la série originale	47
3.23	Restauration de la série originale	48
3.24	code python pour comparer ARIMA Forecast et le réel	48
3.25	Reconstruction de la série d'origine à partir des prédictions	49
3.26	Modèle combinée entre ARIMA(11,1,1) et Random Forest	51

Introduction générale

Dans le secteur bancaire , la gestion de la liquidité constitue un enjeu stratégique majeur. En effet , pour répondre aux besoins de retrait de ses clients, notamment lors de périodes spécifiques comme le Ramadan , l'Aïd ou d'autres événements saisonniers la banque emprunte des montants auprès de la banque centrale . Une anticipation précise des besoins futurs en liquidités est donc indispensable pour éviter à la fois des coûts inutiles liés à un excès de liquidité et des risques d'illiquidité en cas de sous-estimation. L'objectif de ce projet est ainsi de développer des modèles prédictifs basés sur les séries temporelles pour estimer ces besoins à partir des comportements passés des clients.

Les séries chronologiques sont couramment utilisées dans divers domaines notamment la finance l'économie la météorologie et l'ingénierie, pour détecter des tendances, des cycles et des comportements saisonniers . Dans ce cadre , notre projet a pour ambition d'analyser les fluctuations des données, d'identifier les composantes sous-jacentes (tendance, saisonnalité, bruit) et de proposer des modèles de prévision efficaces. Une analyse approfondie des transactions financières a été menée, en utilisant des concepts de modélisation prédictive et d'analyse temporelle, afin de fournir des prévisions précises et utiles pour optimiser la stratégie de trésorerie de la banque.

L'approche adoptée comprendra une analyse exploratoire des données , une étude comparative des différentes méthodes de modélisation et une évaluation rigoureuse des performances des modèles sur la base de critères pertinents (tels que l'erreur de prédiction, la robustesse et l'interprétabilité)

Ce travail permettra ainsi de mieux comprendre les dynamiques temporelles des transactions bancaires et d'améliorer les prises de décision stratégiques en matière de gestion des liquidités .

Chapitre 1

Présentation du Contexte général du projet

1.1 Introduction

Ce chapitre a pour objectif de définir les concepts qui seront abordés dans ce projet de recherche. Il comporte trois parties. Dans la première, nous présenterons le contexte global de notre projet afin de mieux situer la problématique étudiée. La deuxième partie est consacrée à la définition des notions fondamentales en lien avec les séries temporelles, ainsi qu'à la présentation mathématique des tests et méthodes utilisés dans ce cadre. Enfin, la troisième partie s'attache à analyser et modéliser les séries temporelles à partir des résultats du travail effectué, en vue d'une interprétation des résultats.

1.2 Le cadre du projet

Ce projet a pour objectif d'affirmer les points suivants :

- Identifier les périodes de forte activité bancaire (notamment celles consécutives à l'Aïd, au Ramadan...) qui influencent les retraits des clients.
- Modéliser et prédire les besoins en liquidité de la banque à partir des comportements de transaction historiques des clients.
- Permettre à la banque d'anticiper les montants à emprunter auprès de la banque centrale, afin d'assurer une gestion optimale de la trésorerie.
- Évaluer l'efficacité de différents modèles de séries temporelles (ARIMA, modèles saisonniers, etc.) dans ce contexte prédictif.

1.3 Problématique

Comment prédire avec précision les besoins en liquidités d'une banque, à partir des données historiques de transactions, afin d'optimiser les montants empruntés auprès de la banque centrale ? L'enjeu est de prendre en compte la saisonnalité pour éviter le coût d'une mauvaise anticipation tout en disposant des fonds nécessaires..

Conclusion :

Ce projet vise à apporter une réponse méthodique et précise à un enjeu crucial de gestion bancaire : l'anticipation des besoins en liquidités. À travers l'application de modèles de séries chronologiques classiques et avancés, on cherchera à capter la dynamique des transactions financières sur le plan temporel, saisonnier et aléatoire, afin d'élaborer des prévisions fiables, au service d'une gestion optimale des ressources de la Banque, dans le cadre d'une meilleure stratégie de trésorerie.

Chapitre 2

Théorie de la modélisation des Séries Temporelles

2.1 Introduction :

Dans divers domaines tels que la finance, l'économie, la météorologie, etc. Les informations sont fréquemment recueillies sur des périodes de temps. Il est possible d'analyser ces données chronologiques afin de comprendre non seulement les évolutions passées, mais aussi de prévoir les comportements futurs. Avant de procéder aux détails techniques, il est primordial de saisir la signification des séries temporelles et leur importance pour la prise de décision et l'analyse prédictive.

2.2 Définition d'une serie chronologique

Une série temporelle ou série chronologique est l'ensemble des observations d'une variable statistique qui sont réalisées à intervalles de temps réguliers (jours, semaines, mois, trimestres, semestres, années. . .).[1]

L'étude des séries temporelles poursuit plusieurs buts pratiques :

- une meilleure compréhension du phénomène physique représenté par la série
- une représentation simplifiée par un modèle stochastique
- une prédiction du futur de la série à partir de la partie observée.

2.3 Composantes d'une série chronologique

La décomposition d'une série temporelle constitue une méthode statistique qui a pour but de décomposer cette série en plusieurs morceaux, chaque partie représentant une caractéristique particulière de la série : tendance, saisonnalité, cycle et résidus.

2.3.1 Tendance

La tendance T_t correspondant à une évolution à long terme de la série. la tendance majeure d'une série temporelle qui montre la direction dans laquelle se déplacent les données sur le long terme.[2]

par exemple :

- tendance linéaire : $T_t = a + bt$
- tendance quadratique : $T_t = a + bt + ct^2$
- tendance logarithmique : $T_t = \log(t)$

2.3.2 Saisonnalité

La saisonnalité se définit comme la présence d'un phénomène à caractère périodique qui se retrouve tout au long de la série temporelle.

Ainsi, si la composante saisonnière se répète selon période k . On aura $S_{t+k} = S_t$

De nombreuses données présentent des saisonnalités, comme les données météorologiques (les variations de températures dans le temps, la hausse des ventes de billets d'avion en été, etc). [2]

2.3.3 Cycle

La composante de la série fait alors référence à une certaine récurrence mais à une récurrence qui n'est pas périodique c'est-à-dire sur des périodes qui ne sont pas fixes et généralement plus longues, un exemple typique étant le cycle qui correspond aux périodes de récession ou d'expansion. [3]

2.3.4 Résidu

Le résidu du modèle correspond à la partie de la série temporelle qui ne peut pas être expliquée par la décomposition.

Idéalement, ce résidu doit être stationnaire, c'est-à-dire que le processus restant n'évolue pas avec le temps (moyenne et variance constante). Si le résidu de notre série temporelle est non stationnaire, cela signifie que l'on n'a pas réussi à expliquer certaines composantes temporelles dans le modèle. [2]

Remarque : Dans le cadre de l'étude de la disponibilité de liquidité lors des retraits d'argent dans une banque, il est tout à fait pertinent de décomposer la série temporelle en trois composantes : tendance, saisonnalité et résidu. Cette approche est suffisante si l'on observe des comportements réguliers dans les retraits (comme des pics en fin de mois ou certains jours de la semaine) et une évolution progressive du volume global de retraits. La composante cyclique, qui traduit les mouvements à long terme relevant de cycles économiques par exemple, n'apparaît pas ici comme une première nécessité puisqu'on ne la repère pas sur la période considérée ou plus généralement si on est plus intéressé par l'analyse à court ou moyen terme. Cette simplification permet de mieux capter les dynamiques principales tout en gardant un modèle interprétable et efficace pour la prévision opérationnelle.

2.4 Étude de la Stationnarité des Séries Chronologiques : Tests Unitaires et Application du Test de Dickey-Fuller

2.4.1 Stationnarité d'une série chronologique

Un aspect crucial de la série temporelle est sa stationnarité . Si un processus est stationnaire, cela indique que ses caractéristiques statistiques demeurent constantes dans le temps comme sa moyenne , sa variance (homoscédasticité) et sa covariance .[4]

Soit une série temporelle notée à valeurs réelles et en temps discret , X_t est dite stationnaire (stationnarité du second ordre) si : [5]

$$\begin{aligned}\mathbb{E}(X_i) &= \nu, \quad \forall i \in \{1, \dots, t\} \\ \text{Var}(X_i) &= \sigma^2, \quad \forall i \in \{1, \dots, t\} \\ \text{Cov}(X_i, X_{i-k}) &= f(k), \quad \forall i, k \in \{1, \dots, t\}\end{aligned}$$

Si l'une de ces conditions n'est pas vérifiée, on parle donc de non-stationnarité.

Pour la covariance , si on calcule X_2 et X_5 avec ($k=3$) ou entre X_{10} et X_7 avec ($k=3$) , on trouve les memes resultats.

2.4.2 Test unitaire

Il est essentiel de réaliser des tests unitaires sur la stationnarité des séries temporelles car graphiquement il est souvent difficile d'identifier si une série présente une racine unitaire c'est-à-dire si elle est non stationnaire . Les tests visuels, comme les graphiques de séries temporelles peuvent montrer des tendances ou des variations mais ils ne permettent pas de conclure de manière fiable si une série est stationnaire ou non , surtout lorsqu'il s'agit de séries avec des racines unitaires.

1. Équation générale du processus AR(1)

$$Y_t = \phi Y_{t-1} + \varepsilon_t \quad \text{avec } \varepsilon_t \sim \text{bruit blanc } (0, \sigma^2)$$

2. Développement par récurrence

En déroulant l'équation :

$$\begin{aligned}Y_t &= \phi Y_{t-1} + \varepsilon_t \\ &= \phi(\phi Y_{t-2} + \varepsilon_{t-1}) + \varepsilon_t \\ &= \phi^2 Y_{t-2} + \phi \varepsilon_{t-1} + \varepsilon_t \\ &\vdots \\ &= \phi^t Y_0 + \sum_{k=0}^{t-1} \phi^k \varepsilon_{t-k}\end{aligned}$$

Donc on obtient :

$$Y_t = \phi^t Y_0 + \sum_{k=0}^{t-1} \phi^k \varepsilon_{t-k}$$

3. Espérance et variance

On suppose que Y_0 est une constante ou indépendante de ε_t .

Espérance :

$$\begin{aligned}\mathbb{E}[Y_t] &= \phi^t \mathbb{E}[Y_0] + \sum_{k=0}^{t-1} \phi^k \mathbb{E}[\varepsilon_{t-k}] \\ &= \phi^t \mathbb{E}[Y_0] + 0 = \phi^t \mathbb{E}[Y_0]\end{aligned}$$

Cas particuliers :

- Si $|\phi| < 1$, alors $\phi^t \rightarrow 0$ donc $\mathbb{E}[Y_t] \rightarrow 0$.
- Si $\phi = 1$, alors $\mathbb{E}[Y_t] = \mathbb{E}[Y_0]$ (marche aléatoire).
- Si $|\phi| > 1$, alors $\mathbb{E}[Y_t] \rightarrow \infty$ si $\mathbb{E}[Y_0] \neq 0$.

Variance :

$$\begin{aligned}\text{Var}(Y_t) &= \text{Var}\left(\sum_{k=0}^{t-1} \phi^k \varepsilon_{t-k}\right) \\ &= \sum_{k=0}^{t-1} (\phi^k)^2 \cdot \sigma^2 \\ &= \sigma^2 \sum_{k=0}^{t-1} \phi^{2k}\end{aligned}$$

Si $|\phi| \neq 1$, alors :

$$\text{Var}(Y_t) = \sigma^2 \cdot \frac{1 - \phi^{2t}}{1 - \phi^2}$$

Cas particuliers :

- Si $|\phi| < 1$, alors $\phi^{2t} \rightarrow 0$ donc $\text{Var}(Y_t) \rightarrow \frac{\sigma^2}{1 - \phi^2}$: **stationnaire**
- Si $\phi = 1$, alors $\text{Var}(Y_t) = \sigma^2 \cdot t$: **non stationnaire**
- Si $|\phi| > 1$, alors $\phi^{2t} \rightarrow \infty$ donc $\text{Var}(Y_t) \rightarrow \infty$: **série explosive (non stationnaire)**

2.4.3 Test de dickey-Fuller

Le test Augmented Dickey-Fuller joue un rôle essentiel dans l'analyse des séries chronologiques car il constitue un moyen précieux pour évaluer la stationnarité des séries. En effet, savoir si une série temporelle est stationnaire est fondamental pour sa modélisation et sa prévision.[6]

Hypothèses

Le test ADF fonctionne sous deux hypothèses concurrentes :

$H_0 : \gamma = 0 \iff \phi = 1 \iff Y_t$ possède une racine unitaire (non stationnaire)

$H_1 : \gamma < 0 \iff |\phi| < 1 \iff Y_t$ est stationnaire (au sens faible)

Considérons le modèle autorégressif :

$$y_t = \phi y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \text{i.i.d. } \mathcal{N}(0, \sigma^2), \quad y_0 = 0$$

Nous voulons tester l'hypothèse :

$$H_0 : \phi = 1 \quad (\text{racine unitaire})$$

contre :

$$H_1 : |\phi| < 1$$

Statistique de test

La statistique considérée est la statistique t de Student associée à l'estimateur de ϕ , donnée par :

$$t_{\hat{\phi}-1} = \frac{\hat{\phi} - 1}{s \times \left(\sum_{t=2}^T y_{t-1}^2 \right)^{-1/2}}$$

où :

$$\hat{\phi} = \frac{\sum_{t=2}^T y_{t-1} y_t}{\sum_{t=2}^T y_{t-1}^2} \quad \text{et} \quad s^2 = \frac{1}{T-1} \sum_{t=2}^T (y_t - \hat{\phi} y_{t-1})^2$$

On considère le modèle autorégressif d'ordre 1 :

$$y_t = \phi y_{t-1} + \varepsilon_t, \quad t = 2, \dots, T$$

L'estimateur des moindres carrés de ϕ est obtenu en minimisant la somme des carrés des résidus :

$$\text{SCR}(\phi) = \sum_{t=2}^T (y_t - \phi y_{t-1})^2$$

La condition du premier ordre donne :

$$\frac{d}{d\phi} \left[\sum_{t=2}^T (y_t - \phi y_{t-1})^2 \right] = 0 \Rightarrow \sum_{t=2}^T y_{t-1} y_t = \phi \sum_{t=2}^T y_{t-1}^2$$

D'où l'estimateur :

$$\boxed{\hat{\phi} = \frac{\sum_{t=2}^T y_{t-1} y_t}{\sum_{t=2}^T y_{t-1}^2}}$$

Sous l'hypothèse nulle $\phi = 1$

On a :

$$y_t = y_{t-1} + \varepsilon_t \quad \Rightarrow \quad y_t = \sum_{i=1}^t \varepsilon_i$$

Ainsi, (y_t) est une marche aléatoire. On introduit alors le processus empirique normalisé :

$$W_T(r) := \frac{1}{\sigma \sqrt{T}} y_{[Tr]} \xrightarrow[T \rightarrow \infty]{\mathcal{L}} W(r)$$

où $W(r)$ est un mouvement brownien standard sur $[0, 1]$.

Convergence de la statistique

On peut maintenant établir la convergence en loi de $t_{\hat{\phi}-1}$. L'idée centrale s'exprime en réécrivant le numérateur et le dénominateur de $t_{\hat{\phi}-1}$ comme des fonctionnelles de y_t et d'utiliser la convergence fonctionnelle vers le mouvement brownien. Donc on a besoin du théorème Donsker qui établit la convergence en loi d'une marche aléatoire vers un processus stochastique gaussien et on est parfois appelé le théorème central limite fonctionnel.

Ce théorème est une référence pour la convergence en loi de marches aléatoires renormalisées vers un processus à temps continus.

1. Numérateur

$$\hat{\phi} - 1 = \frac{\sum_{t=2}^T y_{t-1}(y_t - y_{t-1})}{\sum_{t=2}^T y_{t-1}^2} = \frac{\sum_{t=2}^T y_{t-1}\varepsilon_t}{\sum_{t=2}^T y_{t-1}^2}$$

Sous H_0 , cette expression est asymptotiquement équivalente à :

$$\frac{\int_0^1 W(r) dW(r)}{\int_0^1 W(r)^2 dr}$$

Or, on a l'identité d'Itô suivante :

$$\int_0^1 W(r) dW(r) = \frac{1}{2} [W(1)^2 - 1]$$

ce qui donne :

$$\hat{\phi} - 1 \approx \frac{\frac{1}{2} [W(1)^2 - 1]}{\int_0^1 W(r)^2 dr}$$

2. Dénominateur

Le terme

$$s \left(\sum y_{t-1}^2 \right)^{-1/2}$$

est asymptotiquement :

$$\frac{\sigma}{\sqrt{T}} \left(\sum_{t=2}^T y_{t-1}^2 \right)^{-1/2} \approx \left(\int_0^1 W(r)^2 dr \right)^{-1/2}$$

3. Conclusion

Ainsi, on obtient la convergence :

$$t_{\hat{\phi}-1} \xrightarrow[T \rightarrow \infty]{\mathcal{L}} \frac{\frac{1}{2} [W(1)^2 - 1]}{\left(\int_0^1 W(r)^2 dr \right)^{1/2}}$$

Graphiquement :

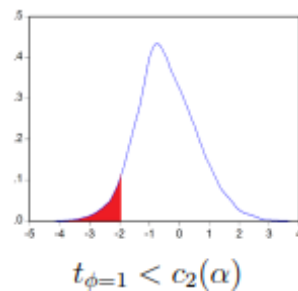


FIGURE 2.1 – la destribution asymptotique[6]

Conclusion

La convergence en loi de la statistique de test vers une fonctionnelle du mouvement brownien implique que sa distribution asymptotique est non standard et asymétrique. Donc pour évaluer la significativité du test de Dickey-Fuller, il est indispensable de comparer la statistique obtenue à des valeurs critiques tabulées spécifiques issues de cette loi limite.

Dans l'analyse des séries temporelles, le test de Dickey-Fuller en particulier dans sa forme augmentée (ADF) est un outil essentiel pour identifier la nature de la non-stationnarité d'une série, notamment en distinguant deux cas fondamentaux : la non-stationnarité de type difference-stationary (DS) et celle de type trend-stationary (TS). Une série DS suit un processus avec racine unitaire (par exemple un AR(1) avec $\varphi = 1$) ce qui implique que la série est non stationnaire mais le devient après une différenciation première :

$$Y_t = Y_{t-1} + \epsilon_t \quad \Rightarrow \quad \Delta Y_t = \epsilon_t$$

Une série TS peut être modélisée comme stationnaire autour d'une tendance déterministe (linéaire ou polynomial) par exemple :

$$Y_t = \mu + \beta t + u_t \quad \text{avec} \quad u_t \text{ stationnaire}$$

Dans ce cas, la série devient stationnaire après soustraction de la tendance (βt) sans nécessiter de différenciation.

Le test de Dickey-Fuller repose sur la réécriture du modèle autorégressif AR(1) sous forme de différences :

$$\Delta Y_t = \gamma Y_{t-1} + \epsilon_t \quad \text{avec} \quad \gamma = \varphi - 1$$

et teste l'hypothèse nulle $H_0 : \gamma = 0$ (racine unitaire donc série non stationnaire) contre l'alternative $H_1 : \gamma < 0$ (stationnarité)

Dans la pratique et pour éviter de mal spécifier la structure déterministe de la série (présence ou absence de constante ou tendance) on applique le test selon une stratégie séquentielle sur trois variantes du modèle :

Modèle général (avec constante et tendance) :

$$\Delta Y_t = \alpha + \beta t + \gamma Y_{t-1} + \sum_{i=1}^p \delta_i \Delta Y_{t-i} + \epsilon_t$$

On teste ensuite la significativité de cette tendance à l'aide d'un test de Student classique (t-test). Si la statistique t associée à β est significative (supérieure en valeur absolue à la valeur critique tabulée de Student à un certain niveau de confiance) alors la tendance est jugée significative ce qui suggère que la série pourrait être trend-stationary . Dans ce cas on applique le test de Dickey-Fuller avec constante et tendance (Case 4 dans le tableau des valeurs tabulées de Dickey-Fuller) pour tester l'hypothèse nulle de racine unitaire.

Modèle avec constante seulement (sans tendance) :

$$\Delta Y_t = \alpha + \gamma Y_{t-1} + \sum_{i=1}^p \delta_i \Delta Y_{t-i} + \epsilon_t$$

Si cette hypothèse est rejetée (la statistique de test est inférieure à la valeur critique) on conclut que la série est stationnaire autour d'une tendance déterministe . En revanche, si la tendance n'est pas significative (le coefficient β n'est pas statistiquement différent de zéro) alors on peut simplifier le modèle en supprimant la tendance et tester la stationnarité en utilisant un modèle avec seulement une constante (Case 2 du test Dickey-Fuller)

Modèle sans constante ni tendance :

$$\Delta Y_t = \gamma Y_{t-1} + \sum_{i=1}^p \delta_i \Delta Y_{t-i} + \epsilon_t$$

Si la constante elle-même s'avère non significative , on teste alors le modèle sans constante ni tendance (Case 1)

Cette progression descendante permet d'éviter les erreurs de rejet ou de non-rejet dues à la mauvaise spécification du modèle (comme inclure à tort une tendance dans une série qui n'en contient pas) et oriente de manière rigoureuse le choix de la transformation à appliquer : retrait de tendance ou différenciation .

Sample size <i>T</i>	Probability that $(\hat{\rho} - 1)/\hat{\sigma}_\epsilon$ is less than entry							
	0.01	0.025	0.05	0.10	0.90	0.95	0.975	0.99
<i>Case 1</i>								
25	-2.66	-2.26	-1.95	-1.60	0.92	1.33	1.70	2.16
50	-2.62	-2.25	-1.95	-1.61	0.91	1.31	1.66	2.08
100	-2.60	-2.24	-1.95	-1.61	0.90	1.29	1.64	2.03
250	-2.58	-2.23	-1.95	-1.62	0.89	1.29	1.63	2.01
500	-2.58	-2.23	-1.95	-1.62	0.89	1.28	1.62	2.00
∞	-2.58	-2.23	-1.95	-1.62	0.89	1.28	1.62	2.00
<i>Case 2</i>								
25	-3.75	-3.33	-3.00	-2.63	-0.37	0.00	0.34	0.72
50	-3.58	-3.22	-2.93	-2.60	-0.40	-0.03	0.29	0.66
100	-3.51	-3.17	-2.89	-2.58	-0.42	-0.05	0.26	0.63
250	-3.46	-3.14	-2.88	-2.57	-0.42	-0.06	0.24	0.62
500	-3.44	-3.13	-2.87	-2.57	-0.43	-0.07	0.24	0.61
∞	-3.43	-3.12	-2.86	-2.57	-0.44	-0.07	0.23	0.60
<i>Case 4</i>								
25	-4.38	-3.95	-3.60	-3.24	-1.14	-0.80	-0.50	-0.15
50	-4.15	-3.80	-3.50	-3.18	-1.19	-0.87	-0.58	-0.24
100	-4.04	-3.73	-3.45	-3.15	-1.22	-0.90	-0.62	-0.28
250	-3.99	-3.69	-3.43	-3.13	-1.23	-0.92	-0.64	-0.31
500	-3.98	-3.68	-3.42	-3.13	-1.24	-0.93	-0.65	-0.32
∞	-3.96	-3.66	-3.41	-3.12	-1.25	-0.94	-0.66	-0.33

FIGURE 2.2 – valeur tabulée de Dickey-Fuller[6]

2.5 Bruit Blanc

2.5.1 Definition

Une série temporelle $X_t, t \in \{1, \dots, n\}$ est un bruit blanc discret si les variables $X_1, X_2, \dots, X_n$ sont Indépendantes (iid) de moyenne égale à 0. tq[7]

- $V(X) = \sigma^2$
- $\rho(X_i, X_j) = 0, \quad \forall i \neq j$

2.5.2 Test de ljung-box

Le test de Ljung-Box du nom des statisticiens Greta M. Ljung et George EP Box , est un test statistique qui vérifie si une autocorrélation existe dans une série chronologique. [8]

Hypothèses

Le test de Ljung-Box utilise les hypothèses suivantes :[8]

H_0 : Les résidus sont distribués indépendamment

H_1 : Les résidus ne sont pas distribués indépendamment ; ils présentent une corrélation en série.

Statistique de test

La statistique de Ljung-Box est donnée par :[8]

$$Q = n(n+2) \sum_{k=1}^h \frac{\hat{\rho}_k^2}{n-k}$$

où :

- n : taille de l'échantillon,
- $\hat{\rho}_k$: autocorrélation empirique au lag k ,
- h : nombre de lags considérés.

Sous l'hypothèse nulle H_0 , la statistique Q suit une loi du chi-deux avec h degrés de liberté.

Justification théorique (sous H_0)

Si les résidus $\hat{\varepsilon}_t$ sont bien un bruit blanc :

Alors, pour chaque k , on a approximativement :

$$\hat{\rho}_k \sim \mathcal{N}(0, 1/n)$$

Donc :

$$n\hat{\rho}_k^2 \sim \chi^2(1)$$

puisque le carré d'une variable normale centrée suit une loi du chi-deux à un degré de liberté.

Ainsi, la somme pondérée :

$$Q \approx \sum_{k=1}^h \chi^2(1) \Rightarrow Q \sim \chi^2(h) \quad (\text{sous } H_0)$$

2.5.3 Test de jarque-bera

Le test de Jarque-Bera est un test d'hypothèse qui cherche à déterminer si des données suivent une loi normale ou non.[9]

Hypothèses

Comme pour chaque test d'hypothèse, il faut poser une hypothèse nulle à valider : [9]

H_0 : les données suivent une loi normale.

H_1 : les données ne suivent pas une loi normale.

La statistique de Jarque-Bera s'écrit :

$$JB = \frac{n-k}{6} \left(S^2 + \frac{(K-3)^2}{4} \right)$$

où :

- n : le nombre d'observations,
- k : le nombre de variables explicatives si les données proviennent des résidus d'une régression linéaire ; sinon, $k = 0$,
- S : le coefficient d'asymétrie (skewness) de l'échantillon testé,
- K : la kurtosis (kurtose) de l'échantillon testé.

Moments empiriques normalisés

Soit $X_1, X_2, \dots, X_n$ un échantillon i.i.d. de loi inconnue. [9]

- **Moyenne** :

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

- **Écart-type** :

$$s = \sqrt{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2}$$

- **Asymétrie (Skewness)** :

$$S_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{X_i - \bar{X}}{s} \right)^3$$

- **Aplatissement (Kurtosis)** :

$$K_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{X_i - \bar{X}}{s} \right)^4$$

Sous l'hypothèse H_0 : normalité

Si $X_i \sim \mathcal{N}(0, 1)$, alors :

$$\mathbb{E}[S_n] \rightarrow 0 \quad \text{et} \quad \mathbb{E}[K_n] \rightarrow 3$$

Par le théorème central limite, on a :

$$nS_n \xrightarrow{\mathcal{L}} \mathcal{N}(0, 6) \quad \text{et} \quad n(K_n - 3) \xrightarrow{\mathcal{L}} \mathcal{N}(0, 24)$$

Ainsi, en normalisant :

$$\frac{nS_n^2}{6} \xrightarrow{\mathcal{L}} \chi^2(1) \quad \text{et} \quad \frac{n(K_n - 3)^2}{24} \xrightarrow{\mathcal{L}} \chi^2(1)$$

Statistique de Jarque-Bera

La statistique est la somme des deux contributions :

$$JB_n = \frac{n}{6}S_n^2 + \frac{n}{24}(K_n - 3)^2$$

Sous H_0 , on a donc :

$$JB_n \xrightarrow{\mathcal{L}} \chi^2(2)$$

puisque c'est la somme de deux variables $\chi^2(1)$ indépendantes.

En effet, on pose $Y = \frac{X_i - \bar{X}}{s}$ alors

$$\mathbb{E}[S_n] = \mathbb{E}[Y^3] = \int_{\mathbb{R}} y^3 \exp\left(-\frac{1}{2}y^2\right) dy$$

Par integration par partie, on obtient

$$\mathbb{E}[Y^3] = 2 \int_{\mathbb{R}} y \exp\left(-\frac{1}{2}y^2\right) dy = 0$$

2.5.4 Test homoscédasticité et hétéroscédasticité

Au sein de la statistique, dans le domaine de la régression linéaire générale, l'homoscédasticité représente une caractéristique essentielle du modèle que l'on inclut parmi les hypothèses de base.

On parle d'homoscédasticité lorsque la variance des erreurs stochastiques de la régression est la même pour chaque observation i (de 1 à n observations). La notion d'homoscédasticité s'oppose à celle d'hétéroscédasticité, qui correspond au cas où la variance de l'erreur des variables est différente. [18]

Hypothèses

$$H_0 : \text{Var}(\varepsilon_i) = \sigma^2 \quad \forall i$$

$$H_1 : \text{Var}(\varepsilon_i) \neq \sigma^2$$

Il existe deux méthodes principales pour décomposer une série temporelle : la décomposition additive et la décomposition multiplicative.[18]

2.6 Décompositions d'une serie chronologique

2.6.1 Décomposition additive

La décomposition additive est utilisée lorsque les variations saisonnières sont relativement constantes au fil du temps. Dans ce cas, la série temporelle (Y) égale à comme la somme de la tendance (T), de la saisonnalité (S) et des résidus (R) :[10]

$$Y_t = T_t + S_t + R_t$$

Graphiquement, l'amplitude des variations est constante autour de la tendance

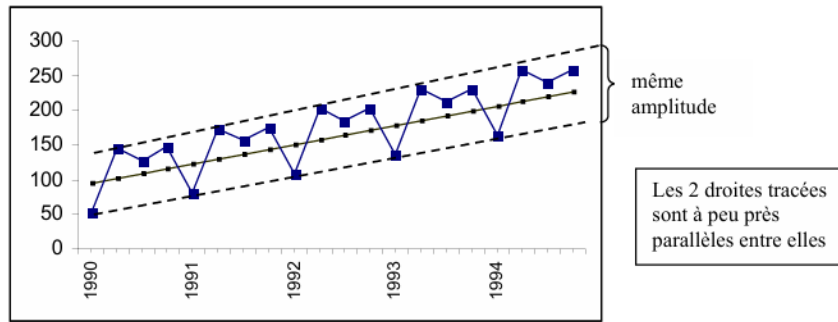


FIGURE 2.3 – Exemple d’une décomposition additive cette figure est importé de [10]

2.6.2 Décomposition multiplicative

La décomposition multiplicative est utilisée quand la variation saisonnière croît ou décroît au cours du temps. Dans ce cas, la série temporelle (Y) est exprimée comme le produit de la tendance (T), de la saisonnalité (S) et des résidus (R) :[10]

$$Y_t = T_t \times S_t \times R_t$$

Graphiquement, l’amplitude des variations (saisonnières) varie.

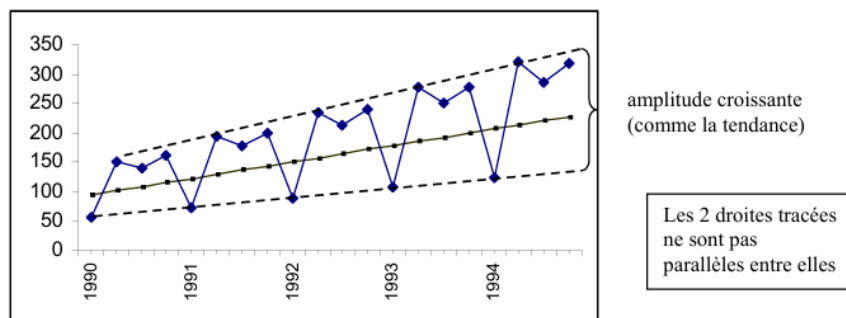


FIGURE 2.4 – Exemple d’une décomposition multiplicative [10]

2.7 Les modèles des Séries Temporelles

Une fois les éléments constitutifs d’une série temporelle examinés et décomposés, nous aborderons les modèles permettant de caractériser et de prédire ces séries. Ces modèles, tels que les modèles ARMA, ARIMA, SARIMA, ARCH et GARCH, utilisent les concepts de tendance, de saisonnalité et de résidus pour prévoir avec précision les données futures. Le choix du modèle dépend des caractéristiques des données et des objectifs de l’analyse.

2.7.1 ARMA

Le modèle ARMA (Auto-Regressive-Moving-Average) est un modèle statistique couramment utilisé pour analyser et prédire les séries temporelles.[11][12]

ARMA est un acronyme pour Auto-Regressive-Moving-Average qui combine :

- Une **composante autorégressive (AR)** d’ordre p , qui exprime la valeur actuelle d’une série en fonction de ses valeurs passées.[2]

Un processus autorégressif d'ordre p , noté $AR(p)$, est un processus stationnaire X_t qui vérifie la relation suivante

$$X_t - \phi_1 X_{t-1} - \phi_2 X_{t-2} - \dots - \phi_p X_{t-p} = \epsilon_t$$

où

$\phi_1, \dots, \phi_p$ sont les coefficients autorégressifs (AR),

ϵ_t est un bruit blanc de moyenne nulle et variance σ^2 .

• **Une composante moyenne mobile (MA)** d'ordre q , qui modélise la dépendance avec les erreurs passées.

un processus moyenne mobile d'ordre q , noté $MA(q)$ est un processus stationnaire X_t qui vérifie la relation suivante

$$X_t = \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \dots - \theta_q \epsilon_{t-q}$$

où

$\theta_1, \dots, \theta_q$ sont les coefficients de la moyenne mobile (MA),

ϵ_t est un bruit blanc de moyenne nulle et variance σ^2 .

conclusion

Une série temporelle X_t suit un modèle $ARMA(p, q)$ si elle vérifie l'équation suivante :

$$X_t = \sum_{i=1}^p \phi_i X_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$$

où

$\phi_1, \dots, \phi_p$ sont les coefficients autorégressifs (AR),

$\theta_1, \dots, \theta_q$ sont les coefficients de la moyenne mobile (MA),

ϵ_t est un bruit blanc de moyenne nulle et variance σ^2 .

p est l'ordre de l'autoregression (AR),

q est l'ordre de la moyenne mobile (MA).

2.7.2 ARIMA

ARIMA, qui est l'une des techniques de prévision de séries chronologiques les plus populaires. ARIMA est populaire parce qu'elle arrive à modéliser efficacement les données de séries temporelles au regard des composantes autorégressives (AR) et de moyenne mobile (MA) mais aussi en tenant compte de la non-stationnarité par la différenciation (I). Cette combinaison rend les modèles ARIMA particulièrement flexibles, c'est pourquoi ils sont utilisés dans des secteurs très différents, comme la finance et les prévisions météorologiques.

Le modèle ARIMA (Auto-Regressive-Integrated-Moving-Average) est une extension du modèle $ARMA(p, q)$ qui permet de modéliser des séries temporelles non stationnaires en les différenciant d fois.[14]

L'équation générale du modèle $ARIMA(p, d, q)$ est :

$$(1 - \sum_{i=1}^p \phi_i B^i)(1 - B)^d X_t = (1 + \sum_{j=1}^q \theta_j B^j) \epsilon_t$$

où :

- p : ordre de la partie AutoRégressive (AR).
- d : nombre de différenciations appliquées pour rendre la série stationnaire.
- q : ordre de la partie Moyenne Mobile (MA).
- B : B est l'opérateur de décalage, qui décale les valeurs de la série temporelle d'un pas en arrière. défini par $BX_t = X_{t-1}$.
- ϕ_i : coefficients de la partie AR.
- θ_j : coefficients de la partie MA.
- ϵ_t : bruit blanc tel que $\mathbb{E}[\epsilon_t] = 0$ et de variance constante σ^2 .

Démonstration :

Le modèle ARIMA(p, d, q) est composé de trois éléments clés :

1- Différenciation pour rendre la série stationnaire (partie intégration I(d))

Certaines séries temporelles présentent une tendance, c'est-à-dire qu'elles ne sont pas stationnaires. Pour corriger cela, on applique une différenciation d'ordre d :

$$(1 - B)^d = (1 - B)(1 - B) \dots (1 - B) \quad (d \text{ fois})$$

Cela revient à soustraire les valeurs précédentes de la série pour obtenir des séries sans tendance.

- pour $d = 0$: la série est déjà stationnaire (on obtient un modèle ARMA(p, q)).
- Pour $d = 1$: on prend la première différence $Y_t = (1 - B)X_t = X_t - X_{t-1}$, ce qui supprime une tendance linéaire.
- Pour $d = 2$, on prend la deuxième différence $Y_t = (1 - B)^2 X_t = X_t - 2X_{t-1} + X_{t-2}$, ce qui supprime une tendance quadratique.

Lorsque $(1 - B)^d$ est appliqué à X_t , on obtient une série stationnaire Y_t , et l'équation devient :

$$(1 - B)^d X_t = Y_t$$

où Y_t est la série transformée stationnaire.

2. Partie AutoRégressive (AR) et Moyenne Mobile (MA)

Une fois la série rendue stationnaire, on applique les parties AR et MA pour modéliser la dépendance de la série avec ses propres valeurs passées et ses erreurs passées.

Partie AR(p) :

Un modèle AR(p) modélise Y_t (la série stationnaire) en fonction de ses p valeurs passées :

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \dots + \phi_p Y_{t-p} + \epsilon_t$$

Cela se traduit par un polynôme dans B :

$$(1 - \sum_{i=1}^p \phi_i B^i) Y_t = \epsilon_t$$

Partie MA(q) :

Un modèle MA(q) modélise Y_t en fonction des q erreurs passées $\epsilon_t, \epsilon_{t-1}, \dots, \epsilon_{t-q}$. Cela se traduit par :

$$Y_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \dots + \theta_q \epsilon_{t-q}$$

Ce qui est représenté par un polynôme dans B :

$$(1 + \sum_{j=1}^q \theta_j B^j) \epsilon_t$$

3. Équation générale ARIMA(p, d, q)

En combinant les trois parties (différenciation d , AR(p) et MA(q)) on obtient l'équation générale du modèle ARIMA(p, d, q) :

$$(1 - \sum_{i=1}^p \varphi_i B^i)(1 - B)^d X_t = (1 + \sum_{j=1}^q \theta_j B^j) \epsilon_t$$

Le terme $(1 - B)^d X_t$ effectue la différenciation pour rendre la série stationnaire.

Le terme $(1 - \sum_{i=1}^p \varphi_i B^i)$ représente la partie auto-régressive (AR).

Le terme $(1 + \sum_{j=1}^q \theta_j B^j)$ représente la partie moyenne mobile (MA)

2.7.3 SARiMA

La méthode SARIMA (saisonnier auto-régression intégrée moyenne Mobile) correspond à une extension du modèle ARIMA prenant en compte la notion de saisonnalité. Elle est particulièrement adaptée à des séries temporelles comportant une signalée saisonnalité.

Dans un modèle ARIMA, nous avons pris en compte la tendance et la différenciation, mais nous n'avons pas pris en compte la saisonnalité.[15]

Le modèle SARIMA ajoute une composante saisonnière à la formulation ARIMA.

La formule générale d'un modèle SARIMA($p, d, q \times P, D, Q_s$) est :

$$1 - \sum_{i=1}^p \phi_i B^i (1 - \sum_{I=1}^P \Phi_I B^{Is}) (1 - B)^d (1 - B^s)^D X_t = (1 + \sum_{j=1}^q \theta_j B^j) (1 + \sum_{J=1}^Q \Theta_J B^{Js}) \epsilon_t$$

où :

- (p, d, q) : paramètres **non saisonniers** du modèle ARIMA
- (P, D, Q) : paramètres **saisonniers** du modèle
- s : période de la saisonnalité
- B : opérateur de retard (backshift), défini par : $BX_t = X_{t-1}$

- ϕ_i : coefficients de la partie AR non saisonnière.
- Φ_I : coefficients de la partie AR saisonnière.
- θ_j : coefficients de la partie MA non saisonnière.
- Θ_J : coefficients de la partie MA saisonnière.
- ϵ_t : bruit blanc (moyenne nulle et variance constante).

conclusion :

Le modèle SARIMA est une version plus flexible du modèle ARIMA qui peut gérer les séries temporelles non stationnaires et saisonnières : une fois que cette série est stationnarisée et que la saisonnalité est prise en compte, le modèle ARMA peut être appliqué pour faire des prédictions. Après avoir étudié les modèles ARMA, ARIMA et SARIMA, qui sont principalement employés pour des séries temporelles stationnaires, nous allons maintenant nous orienter vers les modèles visant à gérer la volatilité dans les séries temporelles. Ces modèles, appelés modèles ARCH et GARCH, revêt une grande importance dans le domaine de la finance.

2.7.4 ARCH

Le modèle ARCH (Autoregressive Conditional Heteroskedasticity) a été introduit par Robert Engle en 1982 pour modéliser la variance conditionnelle des séries temporelles financières, qui présentent souvent une volatilité changeante au fil du temps.[16] Engle a cherché un modèle dans lequel la variance conditionnelle de X_t sachant X_{t-1} dépend de X_{t-1} , et plus particulièrement, $V(X_t|X_{t-1}) = (\alpha + \beta X_{t-1}^2)\sigma$.

Le modèle ARCH(p) est défini par deux équations principales. La première décrit la série temporelle elle-même, tandis que la seconde modélise la variance conditionnelle des erreurs. Pour une série temporelle y_t , le modèle ARCH(p) s'écrit :

$$y_t = \mu_t + \epsilon_t \quad (2.1)$$

où μ_t est la moyenne conditionnelle de y_t et ϵ_t est le terme d'erreur. La variance conditionnelle de ϵ_t , notée σ_t^2 , est modélisée comme suit :

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \alpha_2 \epsilon_{t-2}^2 + \cdots + \alpha_p \epsilon_{t-p}^2 \quad (2.2)$$

où :

- $\alpha_0 > 0$ (terme constant).
- $\alpha_i \geq 0$ pour $i = 1, \dots, p$
- p est l'ordre du modèle ARCH.

2.7.5 GARCH

Un modèle GARCH(p, q) considère la variance conditionnelle σ_t^2 d'une série temporelle y_t comme une fonction linéaire des carrés des erreurs passées et des variances conditionnelles passées. La formulation générale d'un modèle GARCH(p, q) est donnée par :[17]

$$y_t = \mu_t + \epsilon_t \quad (2.3)$$

où μ_t est la moyenne conditionnelle de y_t , et ϵ_t est l'erreur, qui suit une distribution avec une variance conditionnelle σ_t^2 . La variance conditionnelle est modélisée comme suit :

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2 \quad (2.4)$$

où

- $\alpha_0 > 0$: constante (assure que la variance est positive).
- $\alpha_i \geq 0$ pour $i = 1, \dots, q$
- $\beta_j \geq 0$ pour $j = 1, \dots, p$
- q : ordre du terme ARCH.
- p : ordre du terme GARCH.

2.8 Random Forest

L'algorithme de forêt aléatoire est une puissante technique d'apprentissage d'arbre dans l'apprentissage automatique pour faire des prédictions. Ils sont largement utilisés pour les tâches de classification et de régression. Une fois réalisés les prédictions par les différents arbres, la prédiction est obtenue dans les cas de la classification par vote majoritaire et en régression par la moyenne.[21]

Les principales caractéristiques de Random Forest :

- L'algorithme classe les caractéristiques en fonction de leur importance dans la réalisation de prédictions, offrant des informations précieuses pour la sélection et l'interprétabilité des fonctionnalités.
- S'adapte bien aux données volumineuses et complexes sans dégradation significative des performances.
- L'algorithme est polyvalent et peut être appliqué à la fois aux tâches de classification (par exemple, la prédiction de catégories) et aux tâches de régression (par exemple, la prédiction de valeurs continues).

2.9 Métriques d'évaluation

Les métriques d'évaluation nous permettent de quantifier la précision des prédictions réalisées par nos modèles de séries temporelles. Les métriques couramment utilisées incluent :

2.9.1 Mean Squared Error (MSE)

L'erreur quadratique moyenne (MSE) mesure la moyenne des carrés des erreurs entre les valeurs réelles et prédites. Elle est définie comme suit :[23]

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Une MSE plus faible indique une meilleure précision de notre modèle, car les erreurs de prédiction sont plus petites.

2.9.2 Mean Absolute Error (MAE)

L'erreur absolue moyenne (MAE) calcule la moyenne des erreurs absolues entre les valeurs réelles et prédites. Elle est définie comme suit :[22]

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Le MAE nous fournit une mesure intuitive de l'erreur moyenne sans tenir compte de la direction de l'erreur.

2.9.3 Root Mean Squared Error (RMSE)

L'erreur quadratique moyenne racine (RMSE) est la racine carrée de la MSE, offrant une interprétation plus intuitive de l'erreur moyenne des prédictions. Elle est définie comme suit :[25]

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Le RMSE est utile pour comprendre l'ampleur typique de l'erreur de prédiction dans les mêmes unités que les données d'origine.

2.9.4 Coefficient de détermination (R-squared)

Le coefficient de détermination mesure la proportion de la variance des variables dépendantes qui peut être expliquée par le modèle. Il est calculé en comparant la somme des carrés des écarts entre les valeurs réelles y_i et les valeurs prédites $\hat{y}_i$ avec la somme des carrés des écarts entre les valeurs réelles et la moyenne des valeurs réelles $\bar{y}$. La formule est donnée par : [24]

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

où :

- n est le nombre total d'échantillons,
- $\bar{y}$ est la moyenne des valeurs réelles.

Conclusion :

Le but de ce chapitre était de donner une bonne compréhension des séries temporelles et des modèles employés pour les analyser. Grâce à cette compréhension, nous sommes davantage en mesure de saisir les modèles sous-jacents dans nos données et de faire des prédictions précises concernant les données à venir.

Chapitre 3

Analyse et modélisation des Séries Temporelles

3.1 Introduction

Au cours de ce chapitre, nous présentons l'analyse descriptive et visuelle des données de transaction dans le but de dégager les structures temporelles sous-jacentes. Cette étape d'analyse sera décisive pour guider le choix des modèles de prévision les plus appropriés aux typologies observées.

3.2 Présentation des données

Les données utilisées dans cette étude contient 1097 enregistrements répartis sur deux colonnes :

- **Date** : une variable de type texte représentant la date de chaque transaction. Chaque valeur est unique, indiquant une série temporelle quotidienne sans doublons.
- **Montant-Transaction** : une variable numérique (float) représentant le montant des transactions pour chaque date. La moyenne est d'environ 300.66, avec un minimum de

	Date	Montant_Transaction		Date	Montant_Transaction
0	2020-01-01 00:00:00	110.794951	1092	2022-12-28 00:00:00	510.511753
1	2020-01-02 00:00:00	99.320758	1093	2022-12-29 00:00:00	498.222947
2	2020-01-03 00:00:00	116.264681	1094	2022-12-30 00:00:00	513.062207
3	2020-01-04 00:00:00	134.995609	1095	2022-12-31 00:00:00	498.020705
4	2020-01-05 00:00:00	101.075027	1096	2023-01-01 00:00:00	502.433372

FIGURE 3.1 – 5 premières et 5 dernières lignes de la base des données

78.41 et un maximum de 526.54.

Listing 3.1 – Code Python pour le nettoyage du data

```
X = df["Montant_Transaction"]
X.describe()
```

```
count    1097.000000
mean      300.659452
std       114.398162
min        78.412559
25%       187.375909
50%       298.269345
75%       414.780413
max       526.535209
Name: Montant_Transaction, dtype: float64
```

FIGURE 3.2 – description de la base de donnée

3.3 Environnement de travail

Nous avons développé notre travail en utilisant des outils et des langages de programmation écrits par la suite :

3.3.1 Python

Python est le langage de programmation open source le plus utilisé par les développeurs. Nous l'avons utilisée vu sa grande performance et l'existence des bibliothèques qui rendent le travail plus efficace.



FIGURE 3.3 – Logo Python

3.3.2 Anaconda

Nous avons utilisé anaconda présenté par la figure ci-jointe, qui est une distribution de logiciels open source permet la programmation en langages python et R. En effet, c'est une plateforme complète pour les sciences de données. Elle est particulièrement utile dans le domaine de l'intelligence artificielle parce qu'elle offre une variété de bibliothèques pour la préparation, la manipulation, la modélisation et l'évaluation des données.



FIGURE 3.4 – Logo Anaconda

En outre, Anaconda est un ensemble de logiciels contenant de nombreux outils et bibliothèques pour la programmation en Python. Parmi ces outils se trouve Jupyter présenté

par la figure ci-dessous, qui permet de créer et partager des documents interactifs contenant du code, des graphiques et du texte.



FIGURE 3.5 – Logo Jupyter NoteBook

3.4 Prétraitement des données

3.4.1 Nettoyage des données

- **Par colonnes** : La base de données ne contient que deux colonnes pertinentes, à savoir Date et Montant-Transaction. Aucune colonne superflue, comme un identifiant de rang, n'a été détectée, ce qui élimine le besoin de suppression de colonnes inutiles.

- **Par lignes** : Une vérification des doublons a été effectuée sur l'ensemble des enregistrements. Aucune ligne dupliquée n'a été identifiée, garantissant ainsi l'intégrité et la fiabilité des données pour les étapes d'analyse et de modélisation prédictive.

Listing 3.2 – Code Python pour le nettoyage du data

```
df.duplicated().sum()
```

```
np.int64(0)
```

FIGURE 3.6 – resultat Code Python pour le nettoyage du data

3.5 Conversion et réindexation de la colonne de dates

Une étape essentielle lors du prétraitement des données a été la conversion de la colonne de dates, DATE, en format datetime. Cette transformation a été effectuée afin de simplifier les analyses temporelles ultérieures.

Listing 3.3 – Code Python pour la conversion de la date

```
df1 = df[["Montant_Transaction", "Date"]]  
df1.set_index("Date", inplace = True)  
df1
```


Montant_Transaction	
Date	
2020-01-01	110.794951
2020-01-02	99.320758
2020-01-03	116.264681
2020-01-04	134.995609
2020-01-05	101.075027
...	...
2022-12-28	510.511753
2022-12-29	498.222947
2022-12-30	513.062207
2022-12-31	498.020705
2023-01-01	502.433372

1097 rows × 1 columns

FIGURE 3.7 – resultat Code Python pour la conversion de la date

3.6 Visualisation

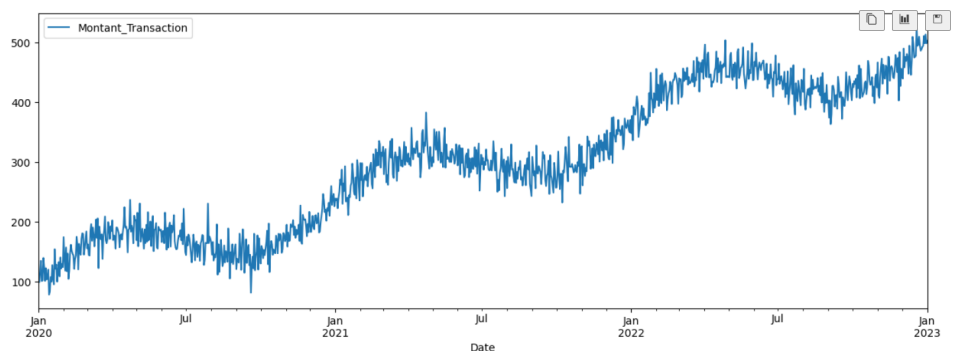


FIGURE 3.8 – Visualisation de la serie chronologique

Interprétation :

La figure représente une série chronologique des montants de transactions bancaires journalières sur une période allant de janvier 2020 à janvier 2023. L'axe horizontal indique la date, tandis que l'axe vertical montre la valeur quotidienne des transactions.

- Une tendance générale à la hausse sur l'ensemble de la période, indiquant une augmentation progressive des montants transigés.
- Des fluctuations saisonnières récurrentes, avec des pics et creux visibles à des intervalles réguliers, probablement en lien avec des événements comme le Ramadan, l'Aïd ou la fin d'année.
- Une variabilité importante à court terme, illustrée par des pointes soudaines dans certaines périodes.

Ces éléments suggèrent la présence de composantes de tendance, de saisonnalité et de bruit, justifiant l'usage de techniques de décomposition et de modélisation temporelle pour améliorer les prévisions.

3.6.1 compareaison de l'évolution des transactions d'une année à l'autre

Les données ont été segmentées en trois périodes annuelles (2020–2021, 2021–2022, 2022–2023) afin de comparer l'évolution des transactions d'une année à l'autre, détecter d'éventuelles variations saisonnières et observer l'évolution de la tendance à court et moyen terme.

Listing 3.4 – Code Python pour la période spécifique 2020-2021

```
df1.loc["2020-01-01":"2020-12-31"].plot(figsize=(15,5))
```

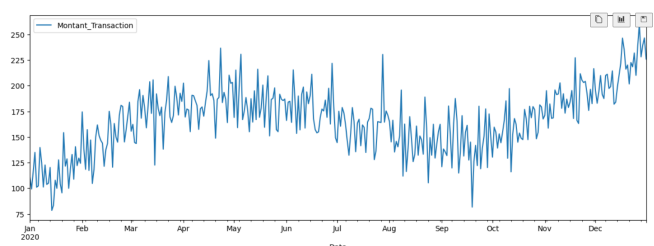


FIGURE 3.9 – filtre une période spécifique 2020-2021

Listing 3.5 – Code Python pour la période spécifique 2021-2022

```
df1.loc["2021-01-01":"2021-12-31"].plot(figsize=(15,5))
```

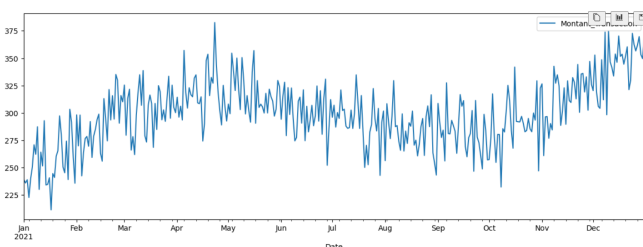


FIGURE 3.10 – filtre une période spécifique 2021-2022

Listing 3.6 – Code Python pour la période spécifique 2022-2023

```
df1.loc["2022-01-01":"2022-12-31"].plot(figsize=(15,5))
```

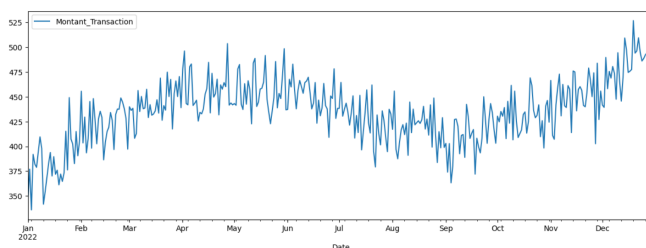


FIGURE 3.11 – filtre une période spécifique 2022-2023

3.6.2 Analyse mensuelle de la distribution des montants

Afin de compléter l'analyse annuelle, nous avons exploré la distribution des montants de transaction par mois à travers un graphique de type boxplot. Ce dernier permet de visualiser les tendances saisonnières potentielles, la variabilité intra-annuelle ainsi que les mois présentant des valeurs atypiques.

Listing 3.7 – Code Python pour la distribution des montants par mois

```
import seaborn as sns
import matplotlib.pyplot as plt
df1['mois'] = df1.index.month
plt.figure(figsize=(12,6))
sns.boxplot(x='mois', y='Montant_Transaction', data=df1)
plt.title("Distribution des montants par mois")
plt.xlabel("Mois")
plt.ylabel("Montant")
plt.show()
```

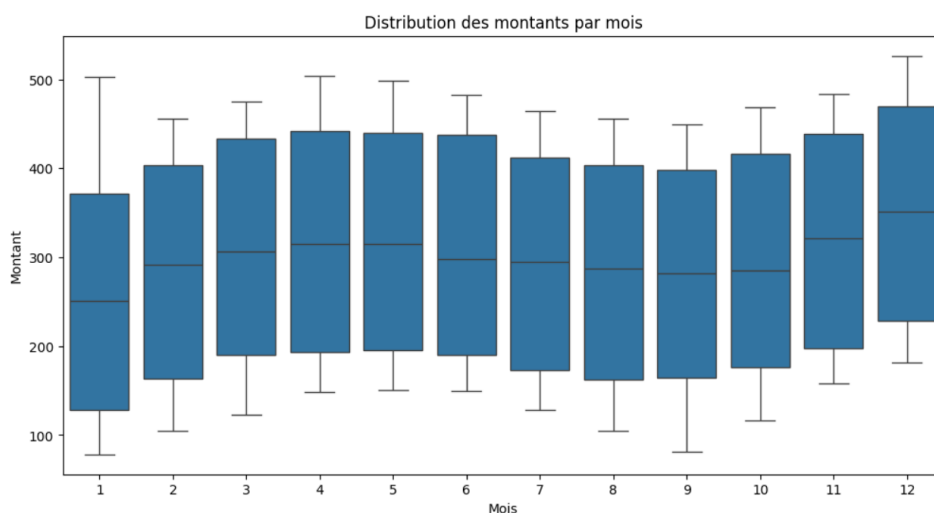


FIGURE 3.12 – Distribution des montants par mois

Interprétation :

On observe une médiane plus élevée en décembre (mois 12) $\Rightarrow$ Cela peut indiquer un pic saisonnier (ex : achats de fin d'année, vacances, primes...).

Janvier (mois 1) montre une médiane plus basse que les autres mois $\Rightarrow$ Possible effet post-fêtes

3.7 Décomposition de la série temporelle

Pour mieux comprendre la dynamique sous-jacente des transactions, la série temporelle a été décomposée en trois composantes fondamentales : la tendance, la saisonnalité et les résidus. Deux approches ont été appliquées :

le modèle additif

Listing 3.8 – Code Python pour le modèle additif

```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt
# Décomposition de la série temporelle
decomposition = seasonal_decompose(df1['Montant_Transaction'],
    ↪ model='additive', period=30)
# Affichage des composantes
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
df1['Montant_Transaction'].plot(ax=axes[0], title='Série
    ↪ originale', color='black')
decomposition.trend.plot(ax=axes[1], title='Tendance')
decomposition.seasonal.plot(ax=axes[2], title='Saisonnalité')
decomposition.resid.plot(ax=axes[3], title='Résidus')
plt.tight_layout()
plt.show()
```

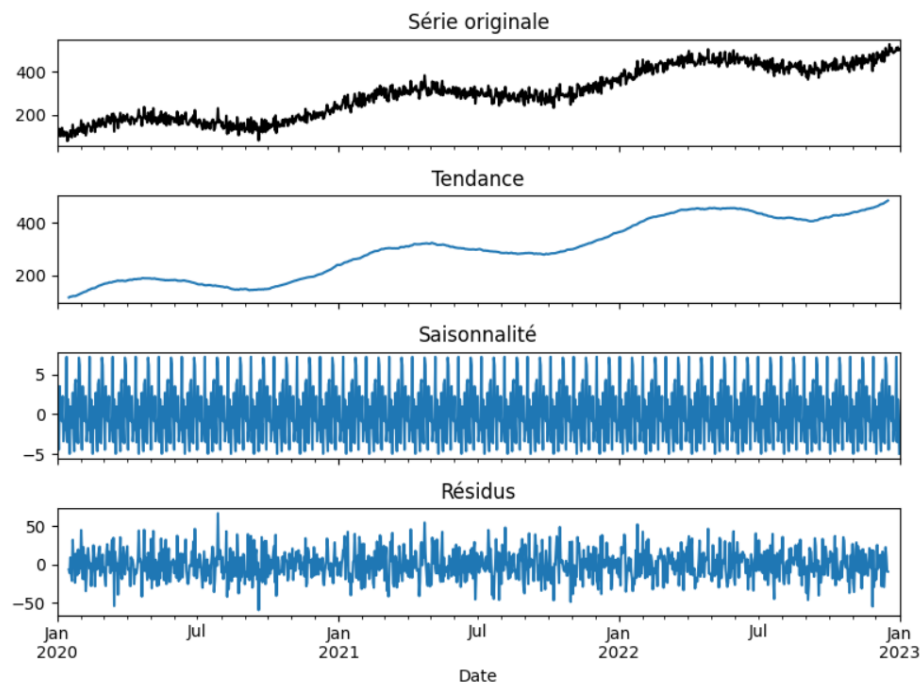


FIGURE 3.13 – Décomposition de la série temporelle selon le modèle additif

Le modèle multiplicatif

Listing 3.9 – Code Python pour le modèle multiplicatif

```
decomposition = seasonal_decompose(df1['Montant_Transaction'],
    ↪ model='multiplicative', period=30)
# Affichage des composantes
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
df1['Montant_Transaction'].plot(ax=axes[0], title='Série
    ↪ originale', color='black')
decomposition.trend.plot(ax=axes[1], title='Tendance')
decomposition.seasonal.plot(ax=axes[2], title='Saisonnalité')
```

```
decomposition.resid.plot(ax=axes[3], title='Residus')
plt.tight_layout()
plt.show()
```

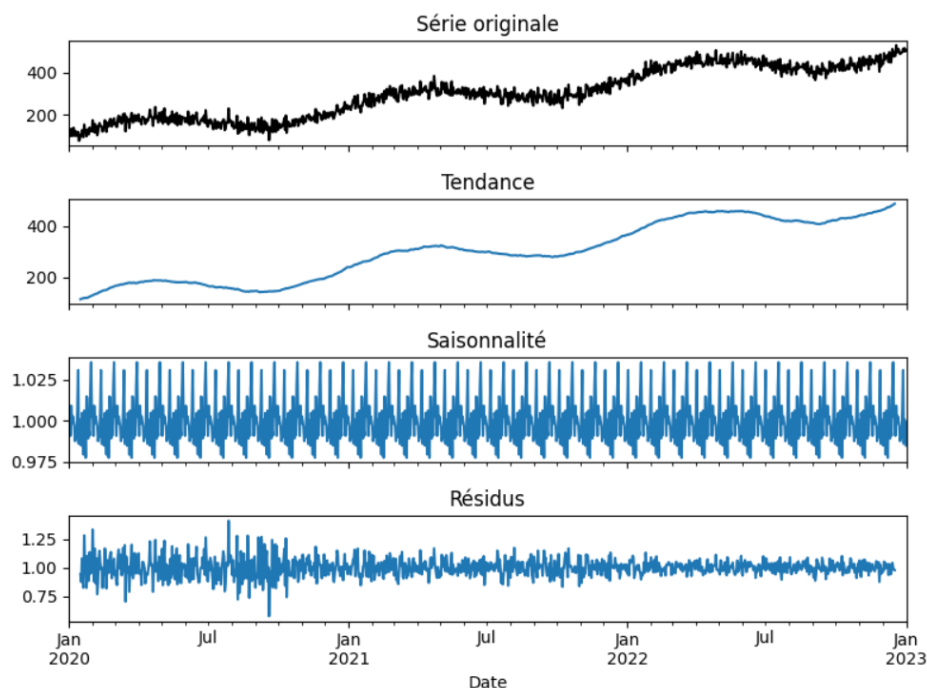


FIGURE 3.14 – Décomposition de la série temporelle selon le modèle multiplicatif

appliquant le logarithme sur le modèle multiplicatif **Le modèle multiplicatif**

Listing 3.10 – Code Python à l'application de logarithme

```
decomposition = seasonal_decompose(np.log(df1['
    ↳ Montant_Transaction']), model='multiplicative', period=30)
# Affichage des composantes
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
df1['Montant_Transaction'].plot(ax=axes[0], title='Serie
    ↳ originale', color='black')
decomposition.trend.plot(ax=axes[1], title='Tendance')
decomposition.seasonal.plot(ax=axes[2], title='Saisonnalité')
decomposition.resid.plot(ax=axes[3], title='Residus')
plt.tight_layout()
plt.show()
```

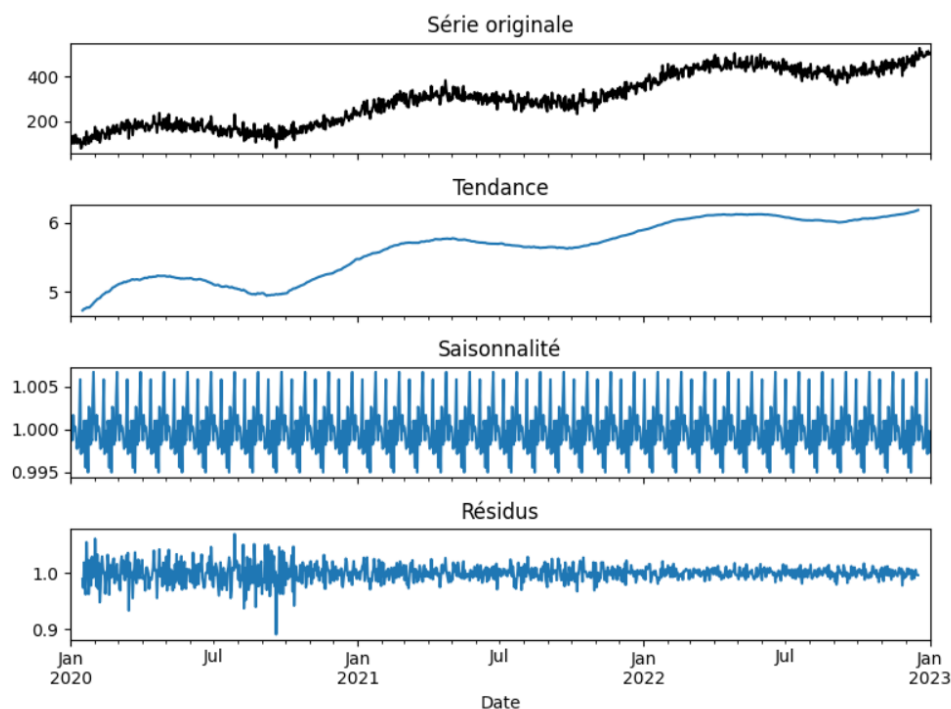


FIGURE 3.15 – Décomposition de la série temporelle en appliquant le logarithme sur le modèle multiplicatif

Interprétation :

- La composante saisonnière observée dans la décomposition présente une variation stable, ce qui respecte l'hypothèse d'amplitude constante.
 - La tendance montre une évolution lente et régulière, bien captée par le modèle.
 - Les résidus sont aléatoires, sans motif clair et centrés autour de zéro, ce qui indique que les principales structures de la série ont été bien captées par la tendance et la saisonnalité.
- ⇒ Notre modèle est additive d'après le graphique de décomposition au niveau de la résidu.

3.7.1 Test ADF

Afin de vérifier si la série temporelle est stationnaire, nous avons appliqué le test statistique Dickey-Fuller Augmenté (ADF).

Ce test est fondamental avant d'appliquer certains modèles comme ARIMA, car ces modèles exigent une série stationnaire. En cas de non-stationnarité, une différenciation peut être nécessaire pour rendre la série stationnaire.

Listing 3.11 – Code Python pour le test ADF

```
# Test statistique plus souvent à utiliser "Dickey-Fuller
  ↳ augmente (ADF) "
from statsmodels.tsa.stattools import adfuller
def test_stationnarite(series):
    result = adfuller(series.dropna())
    print("Statistique de test ADF:", result[0])
    print("P-value:", result[1])
    print("le nombre de retards utilises" , result[2])
    print("nombre d'observations utilisees" , result[3])
```

```
print("Valeurs critiques:", result[4])
print("l information criterion:", result[5])
test_stationnarite(df1["Montant_Transaction"])
# Si p-value > 0.05 : on accepte l hypothese null (H0), c est a
  ↳ dire la serie n est pas stationnaire, .
# Si p-value < 0.05 : on rejette l hypothese null (H0) , c est a
  ↳ dire la serie est stationnaire.
```

```
Statistique de test ADF: -0.330855394769134
P-value: 0.9206859602849251
le nombre de retards utilisés 11
nombre d'observations utilisées 1085
Valeurs critiques: {'1%': np.float64(-3.436391325753478), '5%': np.float64(-2.864207498909067), '10%': np.float64(-2.5681902663042324)}
l'information criterion: 9562.51056631852
```

FIGURE 3.16 – Resultat du test ADF

Interprétation :

La p-value est très élevée (0.92), bien supérieure au seuil de 0.05. Cela signifie que nous ne rejetons pas l'hypothèse nulle (H_0) du test ADF. Par conséquent, la série ne présente pas de stationnarité..

De plus, la statistique ADF (-0.33) est largement supérieure à toutes les valeurs critiques, ce qui renforce cette conclusion.

Dans cette situation, on doit mettre en place une différenciation pour stabiliser la moyenne et supprimer les tendances liées au temps. Cette phase est essentielle pour transformer la série en une série stationnaire.

Listing 3.12 – Code Python pour la différenciation

```
df1['Montant_Transaction'] = df1['Montant_Transaction'].diff()
df1 = df1.dropna(subset=['Montant_Transaction'])
df1['Montant_Transaction'].plot(title="Serie apres
  ↳ differenciation",figsize=(15,5))
```

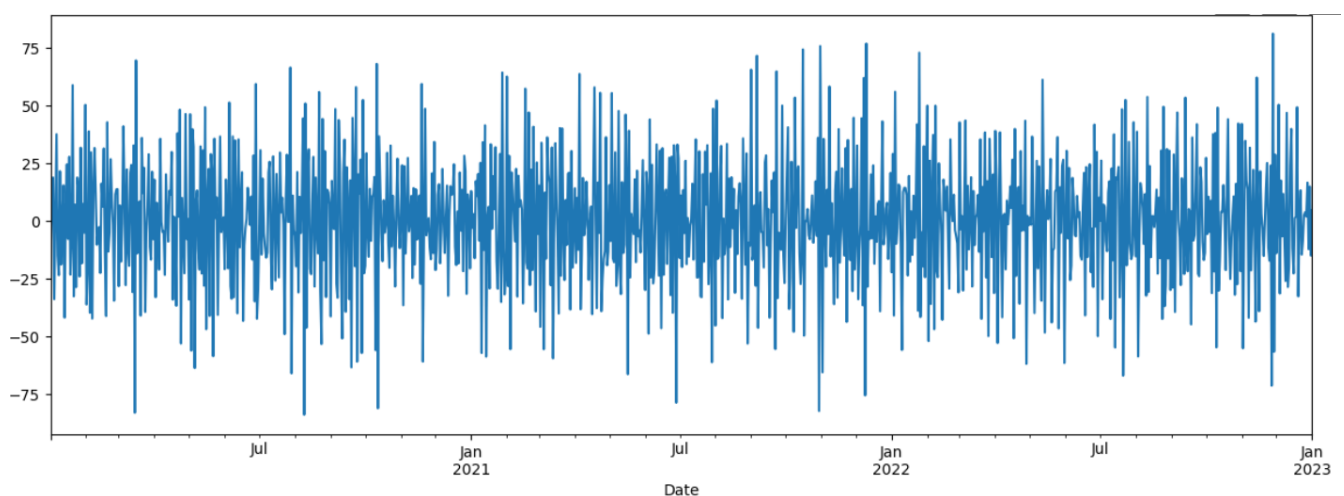


FIGURE 3.17 – série après différenciation

Interprétation :

3.7.2 Test stationnarité

Après avoir appliqué une différenciation à la série temporelle, il est indispensable de réaliser à nouveau un test de stationnarité, comme le test de Dickey-Fuller Augmenté (ADF).

Listing 3.13 – Code Python pour la différenciation

```
test_stationnarite(df1["Montant_Transaction"])
# Si p-value > 0.05 : on accepte l hypothese null (H0), c est a
    ↪ dire la serie n est pas stationnaire, .
# Si p-value < 0.05 : on rejette l hypothese null (H0) , c est a
    ↪ dire la serie est stationnaire.
```

```
Statistique de test ADF: -16.383743807906946
P-value: 2.7357034857701977e-29
le nombre de retards utilisés 10
nombre d'observations utilisées 1085
Valeurs critiques: {'1%': np.float64(-3.436391325753478), '5%': np.float64(-2.864207498909067), '10%': np.float64(-2.5681902663042324)}
l'information criterion: 9552.218760762536
```

FIGURE 3.18 – resultat du test de stationnarité

Interprétation :

La p-value est inférieur au seuil de 0.05. Cela signifie que nous rejetons l'hypothèse nulle (H_0) du test ADF. Par conséquent, la série est stationnaire.

De plus, la statistique ADF (-16.38) est largement inférieure à toutes les valeurs critiques, ce qui renforce cette conclusion.

Conclusion :

il est essentiel de vérifier que la série différenciée ne présente plus de dépendance temporelle significative. Si la série devient stationnaire après une première différenciation, elle peut alors être utilisée dans des modèles adaptés aux séries stationnaires (comme ARIMA).

3.8 Identification et Ajustement des Modèles ARIMA

3.8.1 Estimation des Paramètres ARIMA

Pour déterminer les valeurs de p et q, nous utilisons les graphiques de la fonction d'auto-corrélation (Auto-Correlation Function, ACF) et de la fonction d'auto-corrélation partielle (Partial Auto-Correlation Function, PACF). Ces graphiques aident à identifier les décalages significatifs.

ACF : mesure la corrélation entre la série à différents décalages.

PACF : mesure la corrélation entre la série à différents décalages en contrôlant les décalages intermédiaires. [19]

Listing 3.14 – Code Python pour la différenciation

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 4))
plot_acf(df1["Montant_Transaction"], lags=40)
plt.title("ACF de la serie differenciee (stationnaire)")
```



```
plt.show()
plt.figure(figsize=(10, 4))
plot_pacf(df1["Montant_Transaction"], lags=40)
plt.title("PACF de la serie differenciee (stationnaire)")
plt.show()
```

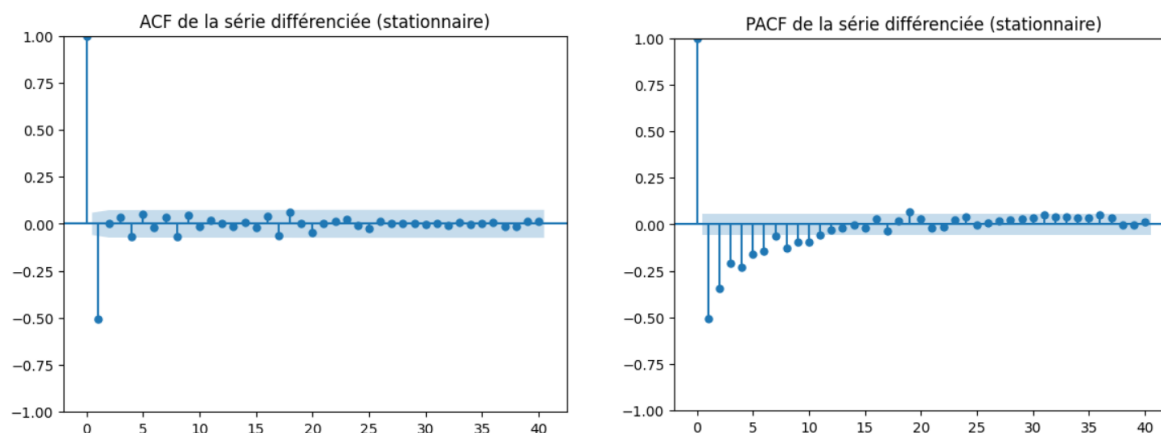


FIGURE 3.19 – ACF et PACF de la série différenciée

Interprétation : L'analyse des corrélogrammes indique la présence d'une composante MA(1), tandis que plusieurs ordres compris entre 6 et 12 seront testés pour la composante AR. Afin de déterminer la configuration optimale du modèle ARIMA(p, 1, 1), nous utiliserons les critères d'information AIC (Critère d'Information d'Akaike) et BIC (Critère d'Information Bayésien), qui permettent de comparer la qualité des modèles tout en tenant compte de leur complexité.

AIC (Akaike Information Criterion) et BIC (Bayesian Information Criterion) sont tous deux des mesures statistiques utilisées dans la sélection de modèles et la modélisation statistique pour évaluer le compromis entre l'ajustement du modèle et la complexité. Ils sont utilisés pour comparer différents modèles et sélectionner celui qui explique le mieux les données.[20]

En général, lorsque l'on compare les modèles utilisant AIC et BIC, des valeurs plus faibles indiquent un meilleur ajustement. Cependant, BIC a tendance à préférer davantage les modèles plus simples qu'AIC. Par conséquent, s'il existe un compromis entre l'adéquation du modèle et la complexité, BIC est plus susceptible de privilégier un modèle plus simple que l'AIC.[20]

Listing 3.15 – Code Python pour déterminer la valeur de p

```
import statsmodels.api as sm
import matplotlib.pyplot as plt
aic_values = {}
bic_values = {}
for p in range(6,15):
    model = sm.tsa.ARIMA(df1['Montant_Transaction'], order=(p, 1,
        ↪ 1))
    results = model.fit()
    aic_values[p] = results.aic
    bic_values[p] = results.bic
    print(f"ARIMA({p},1,1) -> AIC: {results.aic:.2f} | BIC: {
        ↪ results.bic:.2f}")
```

```
plt.figure(figsize=(12,4))
plt.plot(list(aic_values.keys()), list(aic_values.values()),
        ↪ marker='o', label='AIC')
plt.plot(list(bic_values.keys()), list(bic_values.values()),
        ↪ marker='o', label='BIC')
plt.xlabel("Valeur de p")
plt.ylabel("Score AIC / BIC")
plt.title("Sélection du meilleur modèle ARIMA(p,1,1)")
plt.legend()
plt.grid(True)
plt.show()
```

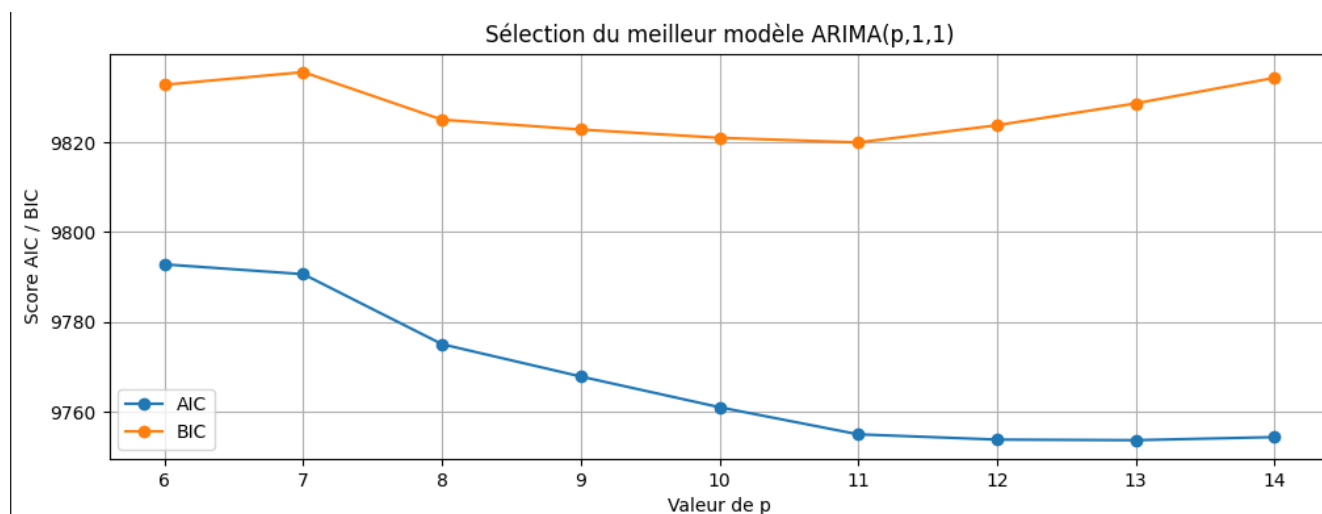


FIGURE 3.20 – sélection de meilleur modèle pour déterminer la valeur de p

Conclusion :

Nous avons évalué que q pourrait être défini comme étant égal à 1. Ces valeurs ont été déterminées en examinant les ruptures significatives dans les graphiques ACF et PACF, ce qui indique le nombre de décalages à prendre en compte pour modéliser correctement la corrélation entre les observations successives de la série temporelle. En fixant p à 11, nous tenons compte de la présence d'une corrélation significative avec une observation à un décalage précédent dans le modèle ARIMA.

L'utilisation d'un modèle SARIMA (ARIMA saisonnier) n'est généralement envisagée que si la série présente une saisonnalité forte non capturée, ce qui se traduirait par :

- des pics réguliers dans l'ACF à des intervalles saisonniers.
- ou une autocorrélation résiduelle significative malgré un ARIMA classique.

Cependant, ce n'est pas le cas.

3.8.2 Validation du modèle ARIMA

Après l'estimation du modèle ARIMA, une phase de validation est essentielle pour s'assurer de sa qualité et de sa capacité prédictive.

Listing 3.16 – Code Python pour validation modèle ARIMA

```
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(df1['Montant_Transaction'], order=(11,1,1))
results = model.fit()
results.plot_diagnostics(figsize=(12, 8))
```

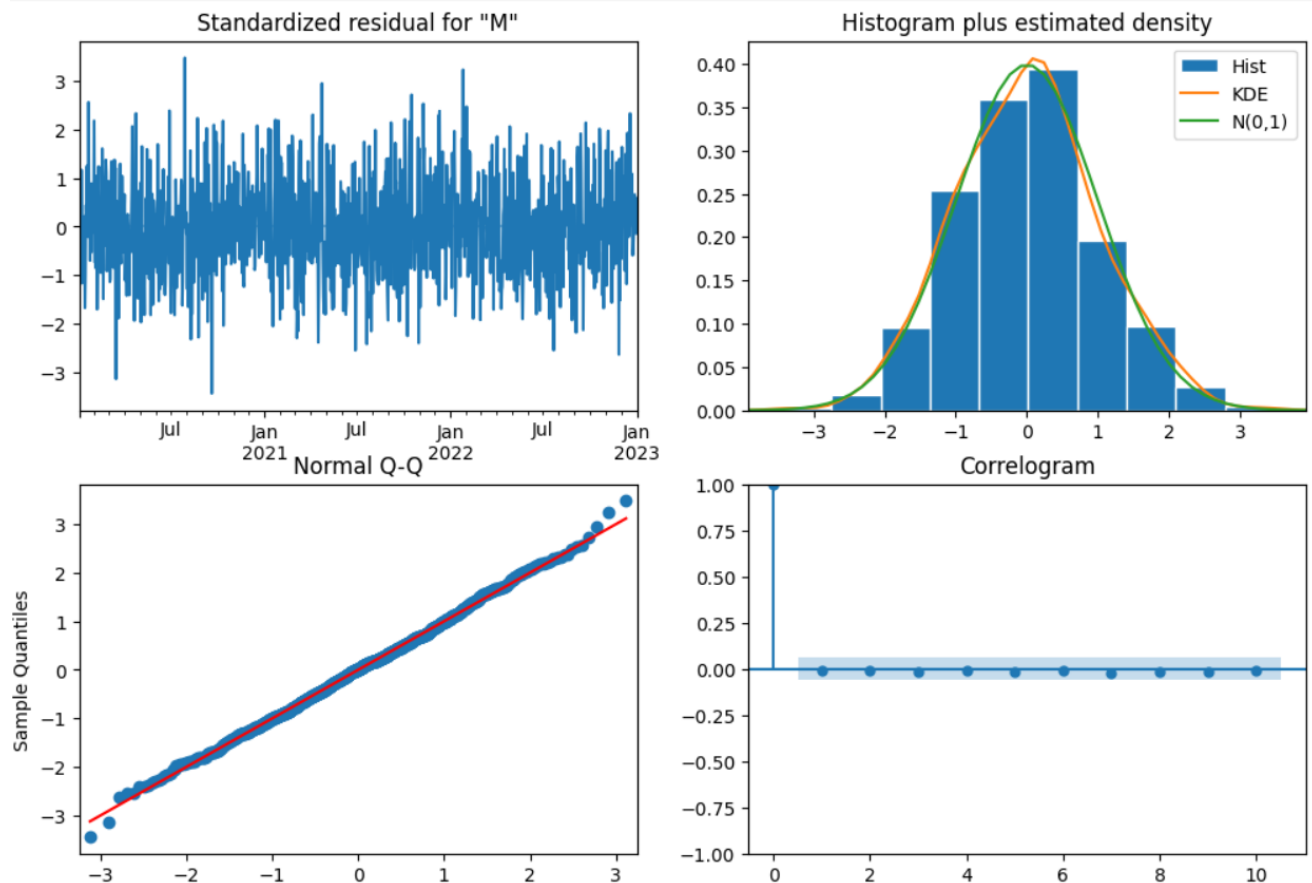


FIGURE 3.21 – Validation du modèle ARIMA

Interprétation :

- Dans la première figure, on observe que la moyenne des résidus est centrée autour de 0, avec une amplitude oscillant entre -3 et 3, ce qui indique une bonne stabilité des erreurs.
- Dans la deuxième figure, les résidus présentent une distribution approximativement symétrique, suggérant une normalité raisonnable.
- Dans la troisième figure (Q-Q plot), la majorité des points suivent une droite diagonale, ce qui indique un bon alignement avec la loi normale.
- Dans la quatrième figure (ACF des résidus), aucune barre ne dépasse l'intervalle de confiance, ce qui signifie qu'il n'existe pas d'autocorrélation significative dans les résidus. Ces observations confirment que les hypothèses du modèle ARIMA sont globalement respectées et que le modèle est bien ajusté.

Afin de compléter la validation du modèle ARIMA, nous allons appliquer le test de Ljung-Box sur les résidus. Ce test permet de vérifier si les résidus du modèle sont indépendants et donc assimilables à un bruit blanc.

Listing 3.17 – Code Python pour test de ljung-Box

```
from statsmodels.stats.diagnostic import acorr_ljungbox
```

```
residuals = results.resid
# Test de Box-Ljung (autocorrelation)
# Ici, on teste jusqu'à un certain nombre de lags, par exemple 11
ljung_results = acorr_ljungbox(residuals, lags=[11], return_df=
    ↪ True)
print("\nBox-Ljung Test (lag=11) :")
print(ljung_results)
if ljung_results['lb_pvalue'].iloc[0] < 0.05:
    print(" Résidus autocorrelés (mauvais ajustement).")
else:
    print(" Pas d'autocorrélation détectée (résidus = bruit blanc
    ↪ ).")
```

```
Box-Ljung Test (lag=11) :
      lb_stat  lb_pvalue
11  5.328978   0.914194
Pas d'autocorrélation détectée (résidus ≈ bruit blanc).
```

FIGURE 3.22 – resultat du test ljung-Box

Interpretation : La p-value élevée (0.91) indique que nous ne rejetons pas l'hypothèse nulle (H_0), selon laquelle il n'existe pas d'autocorrélation significative entre les résidus. Cela confirme que les résidus se comportent comme un bruit blanc, ce qui est un bon indicateur de la qualité de l'ajustement du modèle ARIMA.

Afin de vérifier si les résidus du modèle ARIMA suivent une distribution normale, nous allons appliquer le test de Jarque-Bera.

Listing 3.18 – Code Python pour test de Jarque-Bera

```
from scipy.stats import jarque_bera
# Appliquer le test de Jarque-Bera
jb_stat, p_value = jarque_bera(residuals)
print("Statistique de Jarque-Bera :", jb_stat)
print("P-value :", p_value)
if p_value < 0.05:
    print("Rejet de l'hypothese de normalite des residus.")
else:
    print("On accepte l'hypothese de normalite des residus.")
```

```
Statistique de Jarque-Bera : 1.3285471507215458
P-value : 0.5146472443905301
On accepte l'hypothèse de normalité des résidus.
```

FIGURE 3.23 – resultat du test Jarque-Bera

Interprétation : La p-value est à peu près (0.51), ce qui indique que nous ne rejetons pas l'hypothèse nulle (H_0), selon laquelle les résidus suivent une distribution normale. Cela confirme que la condition de normalité des erreurs est satisfaite, ce qui renforce la validité du modèle ARIMA utilisé.

Afin de vérifier si la variance des résidus du modèle ARIMA est constante dans le temps (homoscédasticité), nous allons appliquer un test d'ARCH (Autoregressive Conditional Heteroskedasticity). Ce test permet de détecter la présence de la hétéroscédasticité.

Listing 3.19 – Code Python pour test d'Hétéroscédasticité

```
from statsmodels.stats.diagnostic import het_arch
# Supposons que vous avez déjà un modele ARIMA ajuste :
residuals = results.resid # resultats de votre ARIMA
# Test ARCH sur les residus
arch_stat, arch_pval, _, _ = het_arch(residuals)
print("Statistique du test ARCH :", arch_stat)
print("P-value :", arch_pval)
if arch_pval < 0.05:
    print(" Heteroscedasticite detectee GARCH recommande.")
else:
    print(" Pas d Heteroscedasticite detectee ARIMA seul est
    ↪ suffisant.")
```

```
Statistique du test ARCH : 3.2045247900968166
P-value : 0.9761927816639944
Pas d'hétéroscédasticité détectée → ARIMA seul est suffisant.
```

FIGURE 3.24 – resultat du test hétéroscédasticité

Interprétation :

La p-value très élevée (0.97) indique que nous ne rejetons pas l'hypothèse nulle (H_0), selon laquelle la variance des résidus est constante (homoscédasticité).

Cela signifie qu'il n'est pas nécessaire d'utiliser un modèle ARCH ou GARCH, car il n'y a pas de preuve d'instabilité de la variance dans les résidus.

3.9 Préviation

Dans cette section, nous nous concentrons sur la prévision des séries temporelles de la liquidité bancaire, en utilisant les modèles économétriques et statistiques validés précédemment. L'objectif est d'estimer les tendances futures de la liquidité, en prenant en compte les fluctuations passées et les relations dynamiques identifiées. La prévision permet de simuler les scénarios futurs, ce qui est essentiel pour une gestion efficace de la liquidité dans le secteur bancaire.

Afin de garantir la fiabilité des prévisions, nous procédons à une division des données en ensembles de formation (train) et de test. L'ensemble de formation est utilisé pour ajuster et calibrer le modèle, tandis que l'ensemble de test sert à évaluer sa performance prédictive. Cette séparation permet de vérifier la capacité du modèle à généraliser les résultats au-delà des données observées, en mesurant l'erreur de prévision sur des données nouvelles.

Listing 3.20 – Séparation des données en ensembles d’entraînement et de test pour la prévision

```
train_size = int(len(df1['Montant_Transaction']) * 0.8)
train_size
train, test = df1['Montant_Transaction'][:train_size], df1[
    ↪ 'Montant_Transaction'][train_size:]
model = ARIMA(train, order=(11, 1, 1))
fitted_model = model.fit()
print(fitted_model.summary())
```

```
-----
Dep. Variable:    Montant_Transaction    No. Observations:      876
Model:            ARIMA(11, 1, 1)        Log Likelihood        -3893.861
Date:             Mon, 12 May 2025      AIC                   7813.722
Time:             00:40:24              BIC                   7875.787
Sample:           01-02-2020            HQIC                  7837.463
                  - 05-26-2022
Covariance Type: opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.9200     0.034    -26.918     0.000     -0.987    -0.853
ar.L2         -0.8300     0.047    -17.640     0.000     -0.922    -0.738
ar.L3         -0.7279     0.055    -13.262     0.000     -0.835    -0.620
ar.L4         -0.7164     0.060    -12.032     0.000     -0.833    -0.600
ar.L5         -0.5966     0.060     -9.894     0.000     -0.715    -0.478
ar.L6         -0.5065     0.062     -8.171     0.000     -0.628    -0.385
ar.L7         -0.4267     0.062     -6.923     0.000     -0.548    -0.306
ar.L8         -0.3906     0.058     -6.721     0.000     -0.504    -0.277
ar.L9         -0.2793     0.054     -5.171     0.000     -0.385    -0.173
ar.L10        -0.1923     0.047     -4.061     0.000     -0.285    -0.099
ar.L11        -0.0890     0.035     -2.517     0.012     -0.158    -0.020
ma.L1         -0.9790     0.008   -115.990     0.000     -0.996    -0.963
sigma2        425.2271    20.804     20.440     0.000    384.452    466.002
...

```

FIGURE 3.25 – Ajustement du modèle ARIMA sur l’ensemble d’entraînement

L’ajustement du modèle ARIMA sur la série temporelle de la liquidité bancaire a permis d’identifier une structure auto-régressive significative. Le résumé statistique du modèle indique que les composantes auto-régressives allant de AR(1) à AR(11) sont toutes significatives, avec des valeurs p-values très proches de 0, ce qui suggère un haut niveau de confiance dans leur contribution au modèle .

Listing 3.21 – code python pour comparer ARIMA Forecast et le réel

```
forecast = fitted_model.forecast(steps=len(test))
plt.plot(test.index, test, label='Test (True)')
plt.plot(test.index, forecast, label='Forecast')
plt.legend()
plt.title('ARIMA Forecast vs Actual')
plt.show()
```

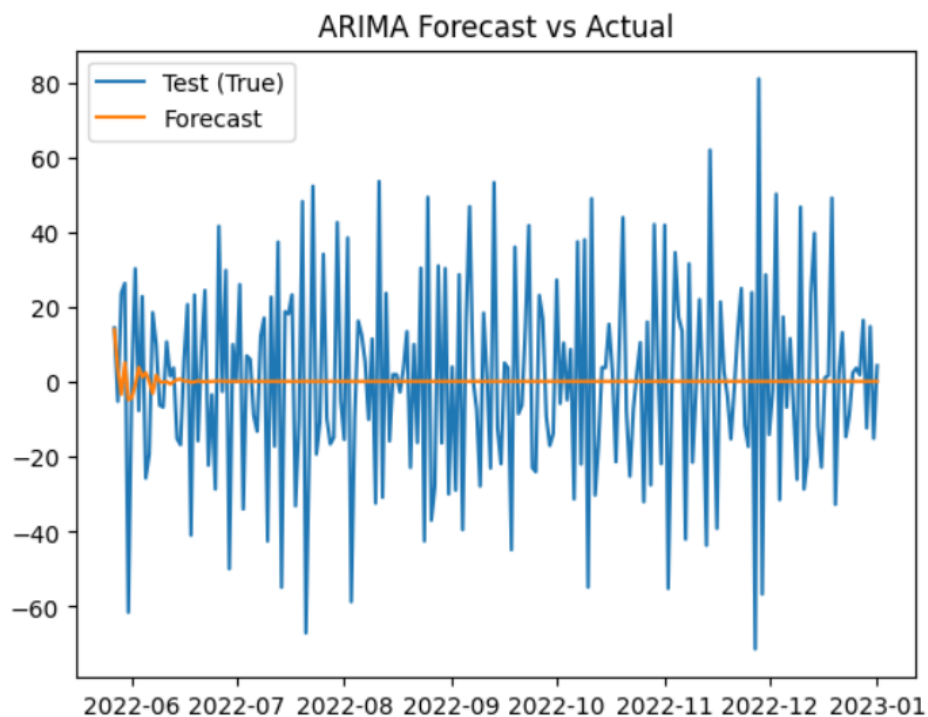


FIGURE 3.26 – ARIMA Forecast

Interprétation :

- La courbe des prévisions (orange) est presque plate, ce qui indique que le modèle a produit des prédictions proches de la moyenne ou constantes.
- En revanche, les valeurs réelles (bleues) présentent une forte variabilité avec de nombreux pics positifs et négatifs.
- Il y a un écart visible entre les prévisions et les observations réelles, suggérant que le modèle n'a pas bien capturé la dynamique de la série, notamment la volatilité et les fluctuations importantes.

Listing 3.22 – Restauration de la série originale

```
ts = df['Montant_Transaction']

train_size = int(len(ts) * 0.8)
train, test = ts[:train_size], ts[train_size:]
model = ARIMA(train, order=(11, 1, 1))

model_fit = model.fit()
start = test.index[0]
end = test.index[-1]
pred = model_fit.predict(start=start, end=end, typ='levels')

rmse = np.sqrt(mean_squared_error(test, pred))
mae = mean_absolute_error(test, pred)
r2 = r2_score(test, pred)

print(f"[ARIMA(11,1,1) - Test set] RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R2: {r2:.2f}")
```

Listing 3.23 – Restauration de la série originale

```
plt.figure(figsize=(12, 6))
plt.plot(ts.index, ts, label='Serie complete', alpha=0.5)
plt.plot(test.index, test, label='Reel (Test)', color='blue')
plt.plot(pred.index, pred, label='Prediction ARIMA', linestyle='
    ↪ --', color='orange')
plt.axvline(x=test.index[0], color='gray', linestyle=':', label='
    ↪ Debut test')
plt.title("Prevision ARIMA(11,1,1)")
plt.xlabel("Date")
plt.ylabel("Montant")
plt.xticks(rotation=45)
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

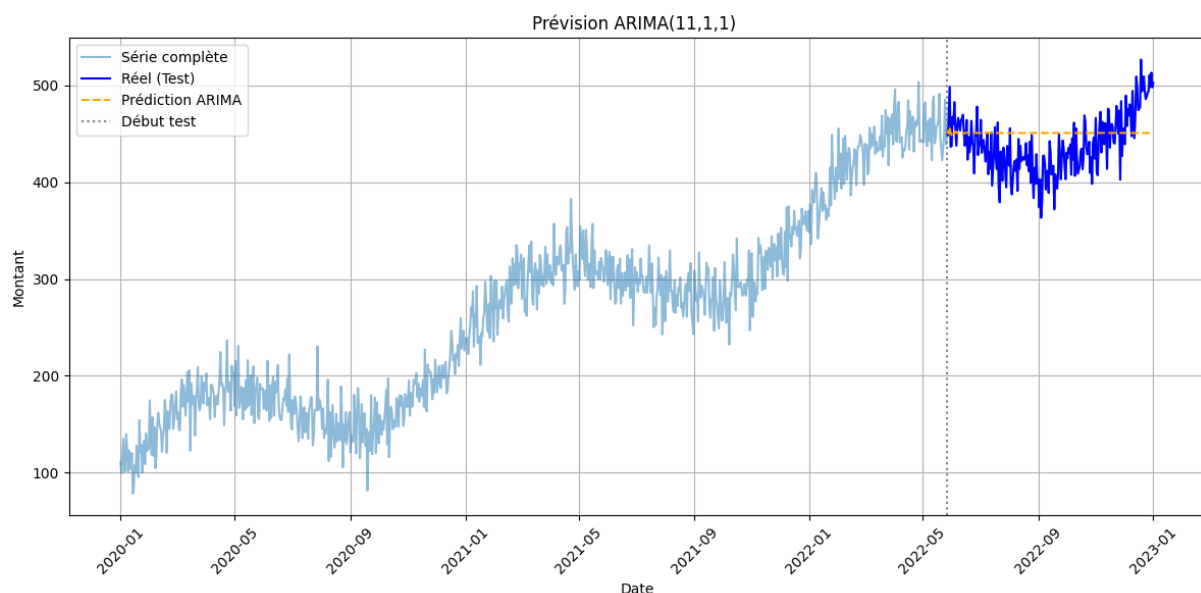


FIGURE 3.27 – Prévion ARIMA(11,1,1)

Interprétation : Le graphique présente les prévisions du modèle ARIMA(11,1,1) appliqué à une série temporelle de montants. La courbe des prédictions suit globalement la tendance des données réelles.

Le modèle montre une bonne capacité à anticiper l'évolution générale, malgré un léger décalage dans la dynamique locale.

Listing 3.24 – code python pour comparer ARIMA Forecast et le réel

```
lags = [1, 2, 3, 4, 5, 6, 7, 30]
for lag in lags:
    df1[f'lag_{lag}'] = df1['Montant_Transaction'].shift(lag)
df1['month'] = df1.index.month
df1['dayofweek'] = df1.index.dayofweek
df1.dropna(inplace=True)
X = df1.drop(columns='Montant_Transaction')
y = df1['Montant_Transaction']
```



```

train_size = int(len(df1) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
plt.figure(figsize=(12,6))
plt.plot(y_test.index, y_test.values, label='Reel')
plt.plot(y_test.index, y_pred, label='Prevision (RF)', linestyle=
    ↪ '--')
plt.title('Prevision Random Forest vs Reel')
plt.legend()
plt.show()

```

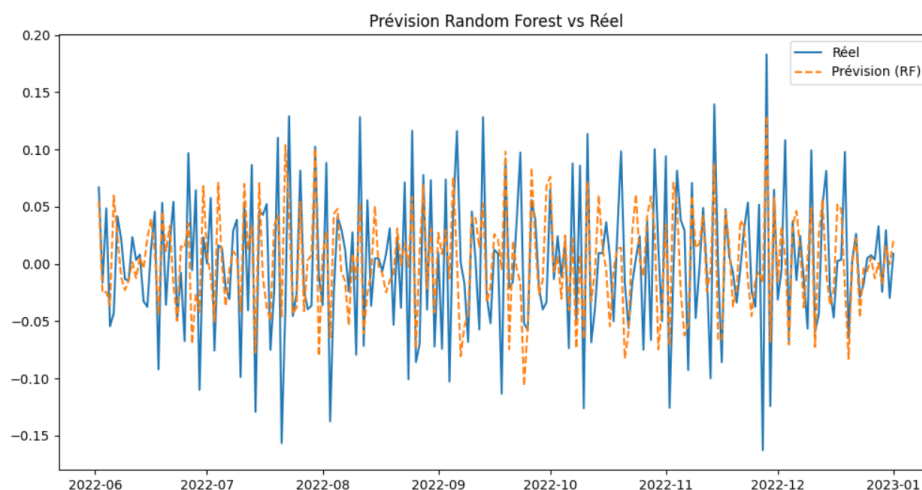


FIGURE 3.28 – Prévision Random Forest

Interprétation :

La courbe de prévision du Random Forest (pointillés orange) suit globalement le profil des valeurs réelles (ligne bleue), capturant les principales variations quotidiennes. Toutefois, le modèle tend à atténuer les extrêmes, sous-estimant parfois les pics positifs ou négatifs. On observe également un léger décalage temporel sur certaines impulsions brusques. Dans l'ensemble, le Random Forest offre une approximation correcte de la dynamique.

Listing 3.25 – Reconstruction de la série d'origine à partir des prédictions

```

valeur_log_initiale = ts_log.loc[X_test.index[0]]
y_pred_log = np.r_[valeur_log_initiale, y_pred].cumsum()[1:]

y_pred_original = np.expm1(y_pred_log)
y_test_original = y_test

rmse = np.sqrt(mean_squared_error(y_test_original,
    ↪ y_pred_original))
print(f"RMSE: {rmse:.2f}")
mae = mean_absolute_error(y_test_original, y_pred_original)

```

```
mse = mean_squared_error(y_test_original, y_pred_original)
r2 = r2_score(y_test_original, y_pred_original)
print(f"MAE: {mae:.2f}")
print(f"MSE: {mse:.2f}")
print(f"R2: {r2:.2f}")

plt.figure(figsize=(12, 6))
plt.plot(y_test.index, y_test.values, label='Reel')
plt.plot(y_test.index, y_pred_original, label='Prevision (RF)',
         linestyle='--')
plt.title("Prevision Random Forest ")
plt.xlabel("Date")
plt.ylabel("Montant de Transaction")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

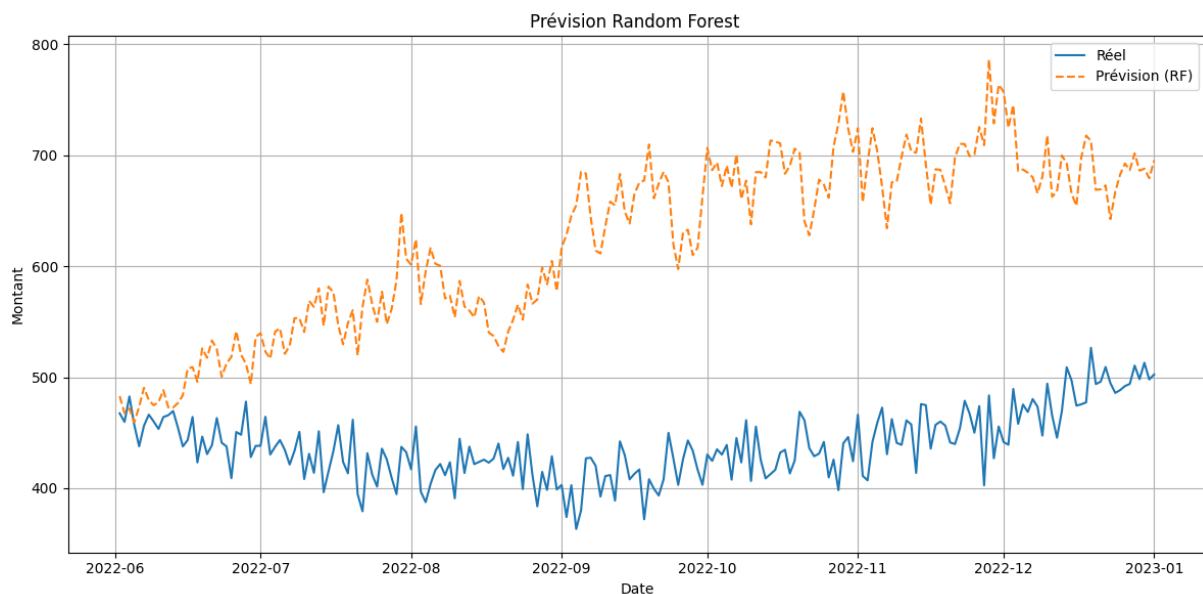


FIGURE 3.29 – Reconstruction de la série d’origine à partir des prédictions

Interprétation :

Ce graphique illustre la comparaison entre les valeurs réelles et les prévisions générées par un modèle Random Forest. On observe que les prévisions suivent la tendance générale de la série, mais restent au-dessus des valeurs réelles. La courbe prédite apparaît plus lissée et moins volatile que la série réelle. Cette reconstruction imparfaite suggère un besoin de recalibrage du modèle ou d’un enrichissement des variables explicatives.

Listing 3.26 – Modèle combinée entre ARIMA(11,1,1) et Random Forest

```
ts = df['Montant_Transaction']
model_arima = ARIMA(ts, order=(11, 1, 1))
model_fit = model_arima.fit()
arima_pred = model_fit.predict(start=12, end=len(ts)-1, typ='
    ↪ levels')
residuals = ts[12:] - arima_pred
residuals = residuals.dropna()

def create_lag_features(series, lags):
    df_lag = pd.concat([series.shift(i) for i in lags], axis=1)
    df_lag.columns = [f'lag_{i}' for i in lags]
    return df_lag
lags = [1, 2, 3, 4, 5, 6, 7, 30]
X = create_lag_features(residuals, lags).dropna()
y = residuals.loc[X.index]
split = int(len(X) * 0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
idx_test = X_test.index
final_pred = arima_pred.loc[idx_test] + rf_pred

y_true = ts.loc[idx_test]
rmse = np.sqrt(mean_squared_error(y_true, final_pred))
mae = mean_absolute_error(y_true, final_pred)
r2 = r2_score(y_true, final_pred)
print(f"[Hybrid ARIMA+RF] RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R2: {r2:.2f}")

plt.figure(figsize=(12, 6))
plt.plot(y_true.index, y_true, label='Reel')
plt.plot(y_true.index, final_pred, label='Prediction ARIMA + RF',
    ↪ linestyle='--')
plt.title("Modele Hybride ARIMA(11,1,1) + Random Forest")
plt.xlabel("Date")
plt.ylabel("Montant")
plt.xticks(rotation=45)
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

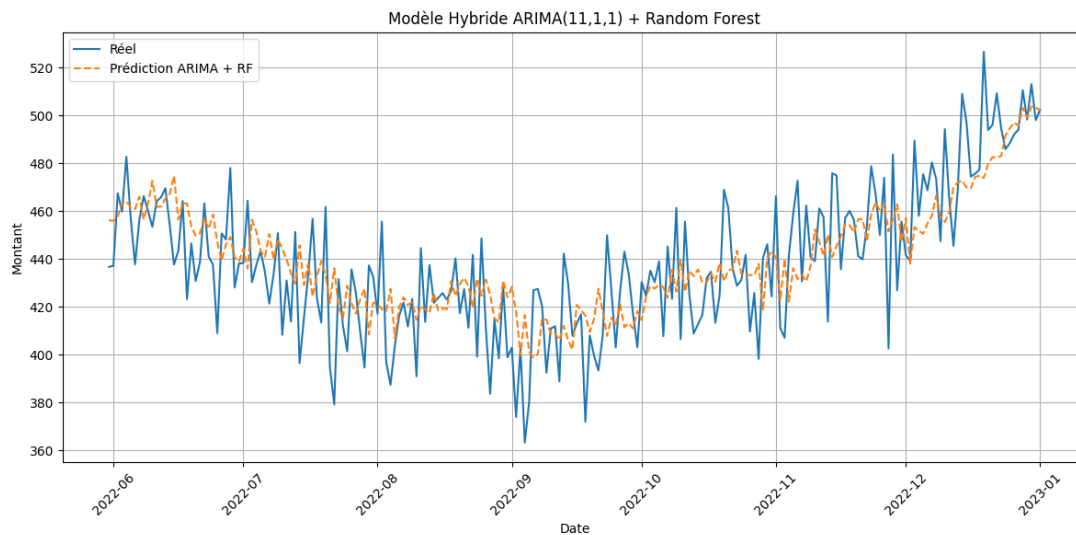


FIGURE 3.30 – Modèle combiné entre ARIMA(11,1,1) et Random Forest

Interprétation :

La figure ci-dessus illustre la performance du modèle hybride ARIMA(11,1,1) combiné avec Random Forest dans la prévision du montant des transactions. La courbe bleue représente les valeurs réelles observées, tandis que la courbe orange en pointillés correspond aux prédictions du modèle. On observe que l'ARIMA capture efficacement la tendance générale de la série, en suivant l'évolution à long terme du signal. Cependant, ARIMA seul ne suffit pas à modéliser les fluctuations irrégulières et les variations locales. L'ajout du modèle Random Forest sur les résidus permet de corriger ces écarts en détectant des motifs non linéaires que l'ARIMA ne prend pas en compte. Cela se traduit dans le graphique par une prédiction qui épouse à la fois la direction globale et les mouvements ponctuels de la série, offrant ainsi une meilleure précision globale.

Conclusion et perspectives

La gestion efficace de la liquidité constitue un défi stratégique majeur pour les institutions bancaires, notamment dans un contexte où les besoins des clients peuvent fluctuer fortement selon les périodes. À travers ce projet, nous avons démontré l'importance d'une modélisation rigoureuse des séries temporelles pour anticiper ces variations et optimiser les décisions de trésorerie. En analysant les transactions passées et en identifiant les composantes essentielles des séries (tendance, saisonnalité, bruit), nous avons pu développer des modèles prédictifs capables de fournir des estimations fiables des besoins en liquidité.

L'approche adoptée, combinant une analyse exploratoire des données et une évaluation comparative des méthodes de modélisation, a permis de mettre en évidence les modèles les plus adaptés aux spécificités des données bancaires étudiées. Les résultats obtenus offrent des perspectives concrètes d'amélioration de la planification financière, en réduisant à la fois les coûts liés à une sur-liquidité et les risques associés à une sous-estimation.

Références

- [1] Série temporelle — Wikipédia, https://fr.wikipedia.org/wiki/S%C3%A9rie_temporelle,
- [2] datascientest, <https://datascientest.com/series-temporelles>,
- [3] CliCours, <https://www.mccours.net/cours/memoires/ahm3clic18.pdf>,
- [4] Marc Stefanon, <https://www.aquiladata.fr/insights/mieux-comprendre-les-methodes-de-prevision-des-series-chronologiques/>,
- [5] Stationnarité d'une série temporelle — Wikipédia, https://fr.wikipedia.org/wiki/Stationnarit%C3%A9_d%27une_s%C3%A9rie_temporelle,
- [6] Ahmed Bensalma, Introduction aux tests de racine unitaire, <https://hal.science/hal-03655798/document>,
- [7] Jean Gaudart, Séries temporelles, https://sesstim.univ-amu.fr/sites/default/files/ressources_pedagogiques/series_temporelles_2.pdf,
- [8] Benjamin Anderson, Test de Ljung-Box : définition + exemple, <https://statorials.org/ljung-box-test/>,
- [9] Test de Jarque-Bera — Wikipédia, https://fr.wikipedia.org/wiki/Test_de_Jarque-Bera,
- [10] Florence NICOLAU, Généralités sur les séries chronologiques, https://webusers.i3s.unice.fr/~crescenz/publications/Florence/introduction-series-chronologiques_chapitre-1.pdf,
- [11] Yannig Goude, Les processus AR et MA, https://www.imo.universite-paris-saclay.fr/~yannig.goude/Materials/time_series/cours5_ARMA.pdf,
- [12] APPRENEZ FACILEMENT LES STATISTIQUES, <https://fr.statisticseasily.com/glossaire/qu%27est-ce-que-ARM-comprendre-les-mod%C3%A8les-ARM-science-des-donn%C3%A9es/#:~:text=Le%20terme%20%C2%AB%20Arma%20%C2%BB%20d%C3%A9signe%20une%20classe,de%20la%20s%C3%A9rie%20en%20fonction%20des%20valeurs%20pass%C3%A9es.>,
- [13] datascientest, <https://datascientest.com/arima-series-temporelles>,
- [14] datacamp, <https://www.datacamp.com/fr/tutorial/arima>,
- [15] Hyndman, R. J., Athanasopoulos, G. (2021). Forecasting : principles and practice (3rd ed.), <https://otexts.com/fpp3/>,
- [16] Jonas Kibala Kuma, MODELISATION HETEROSCEDASTIQUE : LES MODELES ARCH-GARCH, <https://www.jstor.org/stable/1912773?origin=crossref>,
- [17] GARCH, <https://fastercapital.com/fr/contenu/Modeles-GARCH---Modelisation-de-l-heteroscedasticite-dans-les-series-temporelles-1.html#Introduction-aux-mod-les-GARCH>,

- [18] Homoscédasticité — Wikipédia, <https://fr.wikipedia.org/wiki/Homosc%C3%A9dasticit%C3%A9>,
- [19] IBM SPSS Modeler, <https://www.ibm.com/docs/fr/spss-modeler/saas?topic=data-autocorrelation-partial-autocorrelation-functions>,
- [20] AIC vs BIC : différence et comparaison, <https://askanydifference.com/fr/aic-vs-bic/>,
- [21] Random Forest Algorithm in Machine Learning, <https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/>,
- [22] Erreur absolue moyenne — Wikipédia, https://en.wikipedia.org/wiki/Mean_absolute_error,
- [23] Erreur quadratique moyenne — Wikipédia, https://en.wikipedia.org/wiki/Mean_squared_error,
- [24] Coefficient de détermination — Wikipédia, https://en.wikipedia.org/wiki/Coefficient_of_determination,
- [25] Racine carrée moyenne de l'écart quadratique — Wikipédia, https://en.wikipedia.org/wiki/Root_mean_square_deviation,
- [26] Test unitaire — Wikipédia, https://fr.wikipedia.org/wiki/Test_unitaire,